PROFESSIONAL ELECTIVE V

19-202-0808 Blockchain

19-202-0808 BLOCKCHAIN

Course Outcomes:

On completion of this course the student will be able to:

1. *Understand how block chain systems work*
2. *Familarize the functional / operational aspects of cryptocurrency ecosystem*
3. *Design, build and deploy smart contract and distributed applications*
4. *Identify a use case for a blockchain application*

Module I

Introduction – Distributed system – Blockchain and Bitcoin – Types of blockchains – Consensus Decentralization – Decentralization using blockchain – Methods of decentralization – Routes to decentralization – Blockchain and full ecosystem decentralization – Decentralized organizations – platforms

Module II

Introduction – Bit coin – Digital Keys and addresses – Transactions – Blockchain – Mining – alternative coins – alternatives to proof of work – Various stake types – Bitcoin limitations – Smart contracts

Module III

Ethreum – Introduction – Ethereum network – Components of Ethreum ecosystem – EVM – Development tools and Frame woks – Remix – Truffle – Solidity language

Module IV

Hyperledger – Reference architecture Requirements and design goals of hyper ledger fabric – Distributed ledger – proof of Elappsed Time – Transaction families – Attacks on blockchain – Applications on block challenges

References:

1. Imran Bashin, Mastering Block chain, 2nd edition, 2018 packt publishing
2. Aravind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller and steven Goldfeder, Bitcoin and crypto currency Technologies: A comprehensive Introduction, Princeton University Press

Module I

Introduction – Distributed system, Blockchain and Bitcoin – Types of blockchains – Consensus Decentralization – Decentralization using blockchain – Methods of decentralization – Routes to decentralization – Blockchain and full ecosystem decentralization – Decentralized organizations – platforms

Introduction

- Blockchain is a **distributed system** at its core.
- It is a distributed ledger which can be centralized or decentralized.
- A blockchain is originally intended to be and is **usually used as a decentralized platform**.
- It can be thought of as a system that has properties of both decentralized and distributed paradigms.
- It is a decentralized-distributed system.

Distributed System

- A **distributed system** is a collection of independent computers (nodes) connected through a network that share resources, coordinate actions, and appear to users as a single system.
- Each node:
  - Has its own hardware and software resources.
  - Communicates with other nodes via message passing.
  - Participates in performing system-level tasks collectively.
- The **control** in a distributed system may still be centralized (i.e., one central authority can manage it), but **computation and data storage** are physically distributed across multiple nodes.

Distributed System

- **Example – Google Cloud (Distributed but Centralized)**
- Google Cloud runs applications on multiple data centers around the world.
  - Data and processing are distributed across many servers (distributed nature).
  - However, **Google** controls all these servers, configurations, and permissions (centralized control).
- Hence, **it is a distributed but not decentralized system.**

Decentralized System

- Decentralization refers to transferring **control and decision-making** from a single centralized authority to a **distributed network** of participants.
- In a decentralized system:
  - **No single entity owns or governs** the entire network.
  - Every participant can verify, contribute, and validate data independently.
  - System integrity is maintained through **consensus** among nodes.
- **Example – Bitcoin Network (Distributed and Decentralized)**

Electronic cash

- The evolution of electronic cash, cryptographic innovations, and distributed systems leading to blockchain technology
- Two fundamental e-cash system issues need to be addressed: accountability and anonymity.
  - **Accountability** is required to ensure that cash is spendable only once (double-spend problem) and that it can only be spent by its rightful owner. Double spend problem arises when same money can be spent twice. As it is quite easy to make copies of digital data, this becomes a big issue in digital currencies, can make many copies of same digital cash.
  - **Anonymity** is required to protect users' privacy. As with physical cash, it is almost impossible to trace back spending to the individual who actually paid the money.

History of blockchain and Bitcoin

1982 David Chaum proposes e-Cash concept

- Introduced the idea of **digital cash** that could preserve user privacy and prevent double spending using cryptographic techniques.
- Published in his paper *"Blind Signatures for Untraceable Payments."*

1985–1990 Development of Blind Signatures & Secret Sharing

- Chaum and collaborators developed **blind signatures** (for anonymity) and **secret sharing** (for accountability) — foundational ideas for digital payment systems.

1990s Early Digital Currency Experiments

- Several electronic cash systems were tested: **DigiCash**, **e-Gold**, and **Hashcash**.
- However, all relied on **centralized authorities**, which led to trust and regulatory issues.

History of blockchain and Bitcoin

1997 Adam Back introduces Hashcash

- A proof-of-work system designed
- The **Hashcash algorithm** later influenced Bitcoin's **Proof-of-Work** mechanism.

2004 Hal Finney develops Reusable Proof of Work (RPoW)

- Improved on Hashcash by enabling tokens validated by PoW to be reused securely—an important conceptual step toward decentralized money.

Crisis and the White Paper - Bitcoin

2007–2008 Global Financial Crisis (U.S. Subprime Mortgage Collapse)

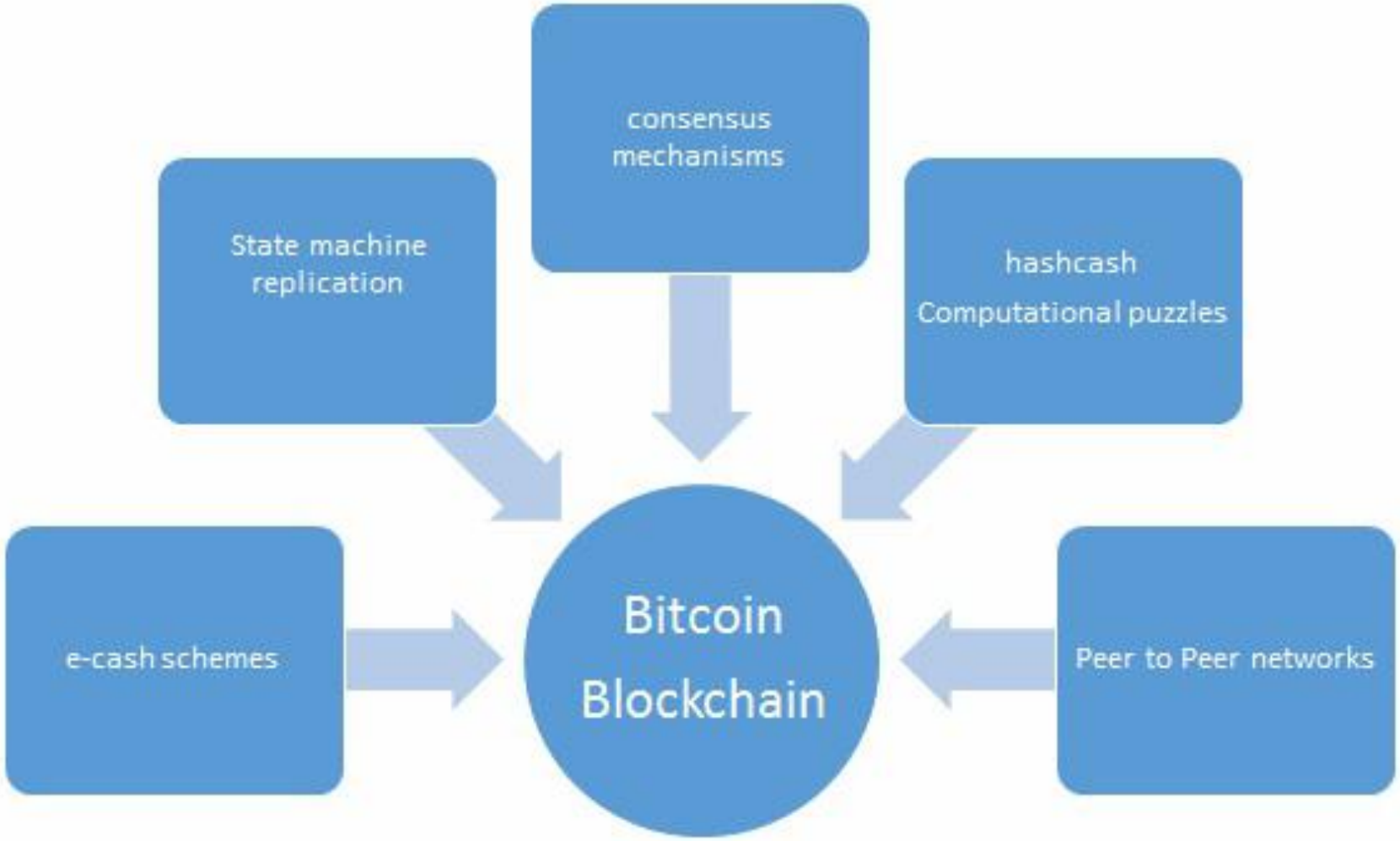
- The **2008 U.S. financial crisis** exposed major flaws in the global banking system — centralized control, excessive speculation, lack of transparency, and government bailouts.
- Public trust in centralized financial institutions eroded dramatically.

Bitcoin's Purpose as a Solution

Bitcoin was created to offer an alternative that directly addresses the failings exposed by the 2008 crisis:

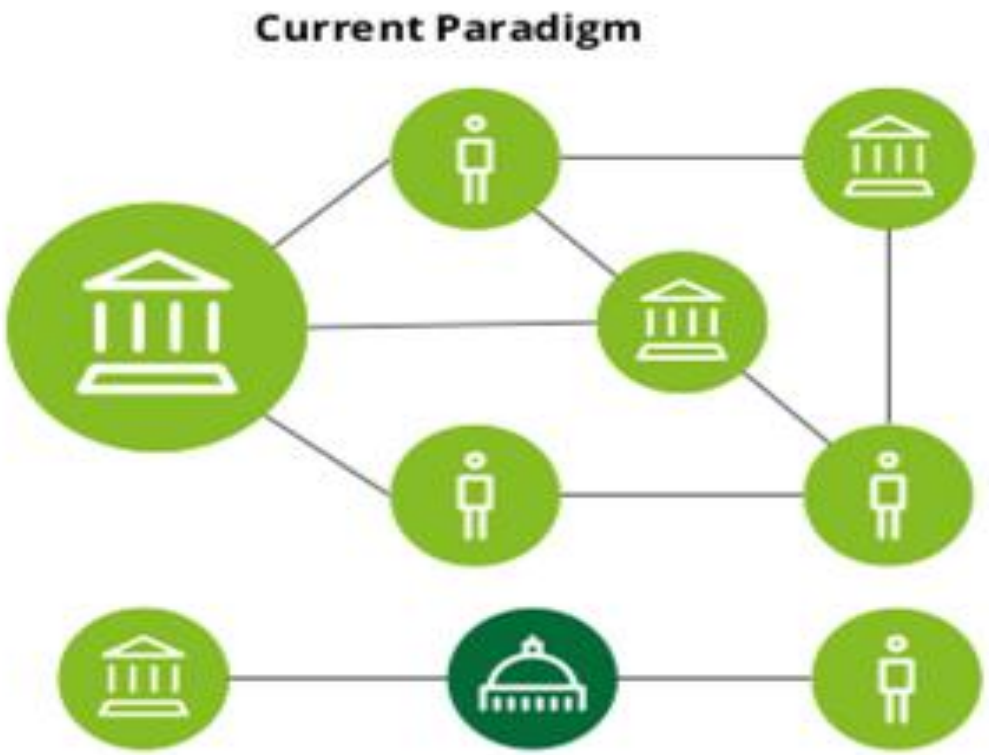
Problem in 2008 Crisis	Bitcoin's Solution (Decentralization)
Single Point of Failure (Banks/Govt)	**Decentralized Network** (No single entity controls it)
Need for "Trust" in fallible institutions	**Cryptographic Proof** (Trust is replaced by code/math)
Bailouts funded by taxpayers	**Limited Supply** (21 million coins, no arbitrary inflation)
Lack of Transparency in financial dealings	**Public Ledger** (The blockchain records all transactions)

Ideas supported the invention of Bitcoin and blockchain



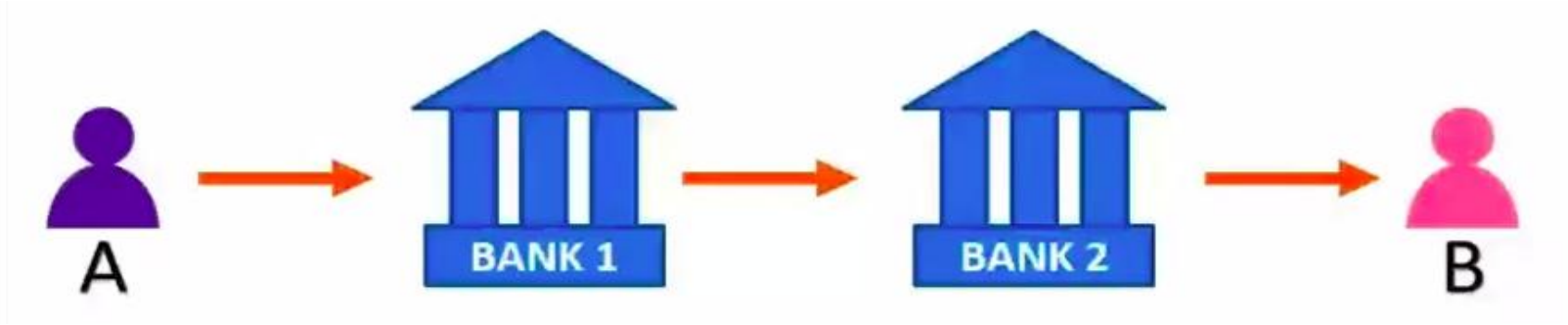
What is blockchain?

- Technology behind the cryptocurrency Bitcoin
- Bitcoin originated from the mistrust in Banks and large institutions
  - Banks keep ledgers but not accessible fully to others
  - We have to trust the banks



- Central authority keeps record of who owns what and when on their system
- Central authoriteis transfer actual value between two parties
- Multiple intermediaries required to facilitate of assets and create trust

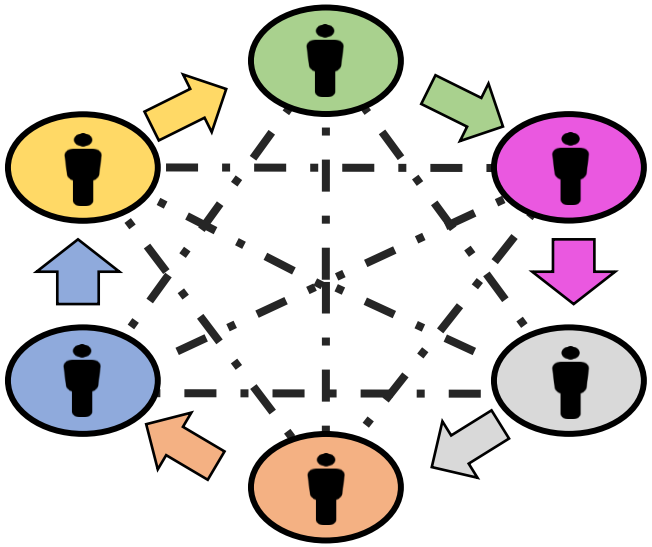
Banking Transaction



- If any of the step fail, the money could vanish. There is a long process of reconciliation to correct the mistake

Blockchain

- Blockchain is a **peer-to-peer distributed ledger** that is **cryptographically secure**, **append-only**, **immutable**, and **updated through consensus** among peers.



A distributed ledger that allows users to verify transactions without an intermediary

Distributed :- no single authority
Ledger :- record of transaction

Blockchain Transaction

- Distributed nodes that maintain a shared source of information
- Trust enabled by cryptographic algorithm

- Alice send transaction “send money to Bob” to the network
- Participants of the network validates the transaction and updates the ledger
- Bob sees that the balance is updated on the ledger
- If any of the steps fail, money will still be in Alice’s account. No reconciliation required

bitcoin

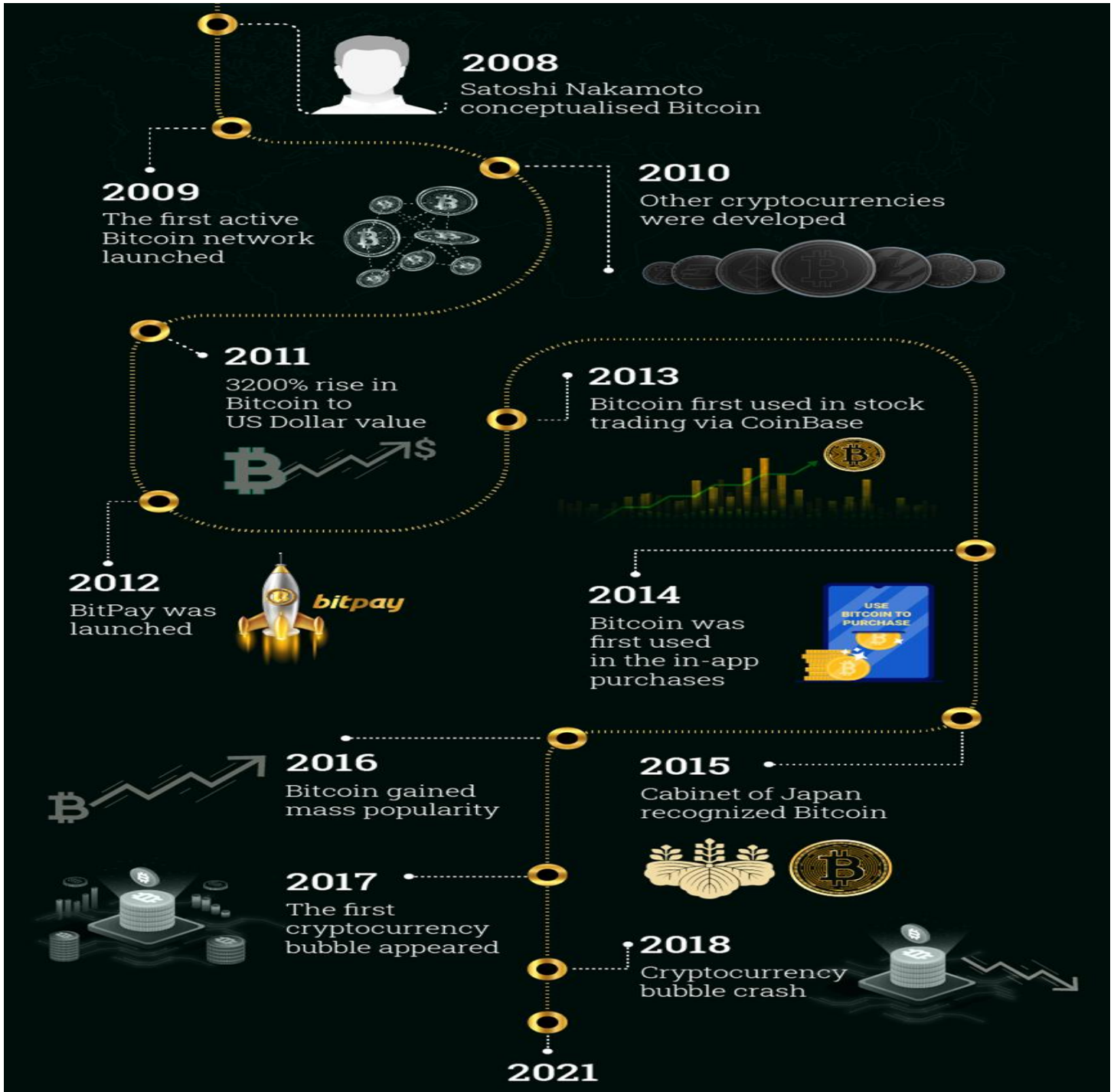
- The idea was published in 2008 by an pseudonymous person/group of people, named Satoshi Nakamoto

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

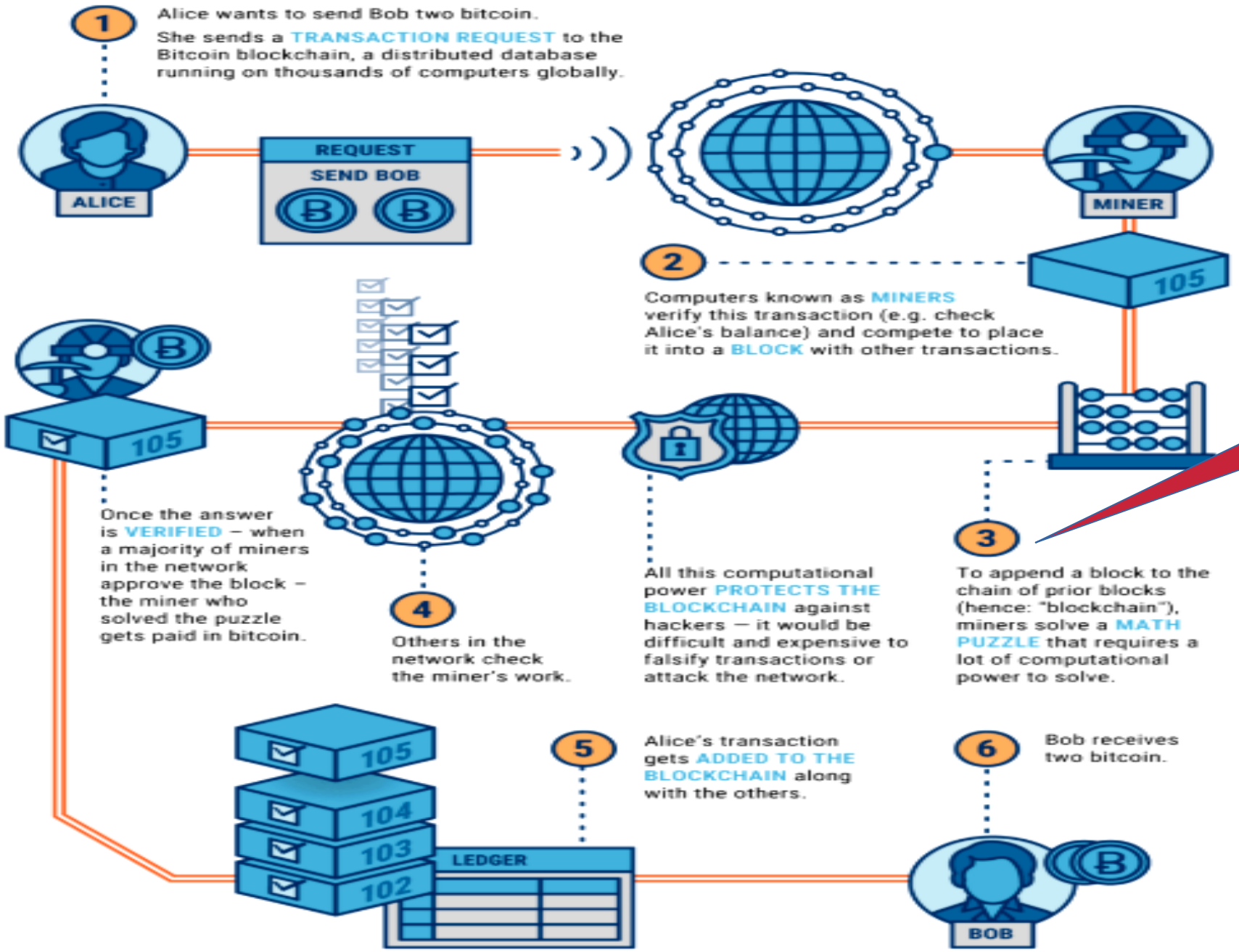
Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

A Brief History of Bitcoin



Understanding a bitcoin transaction

HOW BLOCKCHAIN TECHNOLOGY POWERS BITCOIN



Consensus Mechanism
Bitcoin Network
uses PoW

How blocks get added to a Blockchain?

1. Transactions submitted to network
2. Nodes verify transactions
3. Bundle the transactions into a block
4. Select the header of most recent block and insert it into the new block as hash
5. Consensus algorithm
6. New block is added to the local blockchain and propagated to the network

Types of cryptocurrency

Watchlist | Portfolio | Cryptocurrencies | Categories | DeFi | NFT | Play To Earn | Polkadot | BSC | Solana | Avalanche | Show rows 100 | Filters | Customize

#	Name	Price	24h %	7d %	Market Cap	Volume(24h)	Circulating Supply	Last 7 Days
1	Bitcoin BTC Buy	$64,008.26	2.69%	17.17%	$1,205,727,435,392	$37,423,184,661 585,026 BTC	18,848,781 BTC	
2	Ethereum ETH Buy	$3,860.28	1.31%	12.47%	$454,432,537,223	$15,462,225,711 4,014,957 ETH	117,998,991 ETH	
3	Binance Coin BNB Buy	$486.12	1.45%	8.35%	$81,042,755,915	$1,608,341,296 3,310,267 BNB	166,801,148 BNB	
4	Cardano ADA	$2.11	1.38%	0.57%	$69,288,016,898	$3,217,537,720 1,527,738,988 ADA	32,899,071,908 ADA	
5	Tether USDT Buy	$1.00	0.04%	0.04%	$69,051,121,484	$58,949,859,835 58,943,020,252 USDT	69,043,109,914 USDT	
6	XRP XRP	$1.10	0.78%	1.60%	$51,642,498,150	$2,690,314,837 2,442,114,393 XRP	46,878,114,887 XRP	

- DATA SOURCE: https://coinmarketcap.com/ *AS OF 20/10/2021

Some of the Bitcoin Exchange/Wallet in India

Crypto coins vs. tokens

- A digital coin is created on its own blockchain and acts in much the same way as traditional money
- It can be used to store value and as a means of exchange between two parties doing business with each other
- Examples of coins include Bitcoin and Litecoin
- Tokens, on the other hand, have far more use than just digital money
- Tokens are created on top of an existing blockchain and can be used as part of a software application (like to grant access to an app, to verify identity, or to track products moving through a supply chain), or they can even represent digital art
- Ether is an example of a token, which is used to make transactions on the Ethereum network

Breaking Down the Technical Definition

1. Peer-to-Peer (P2P)

- No central authority; all nodes communicate directly.
- Each node acts as both a client and a server.
- Enables *direct exchange of value* between users (no bank or intermediary).
- **Example:** Bitcoin allows users to send digital money directly.

2. Distributed Ledger

- Every participant has a full copy of the ledger.
- When a new transaction occurs, all copies are updated and verified.
- Prevents single-point failure and ensures data redundancy.
- **Example:** Each Bitcoin node stores the full transaction history.

3. Cryptographically Secure

- Blockchain uses **hashing**, **digital signatures**, and **encryption** to protect data.
- Ensures:
  - **Integrity** (data cannot be altered undetected)
  - **Authentication** (verifies sender)
  - **Non-repudiation** (sender cannot deny transaction)
- **Real-world analogy:** Like a tamper-proof digital seal on every transaction.

4. Append-Only

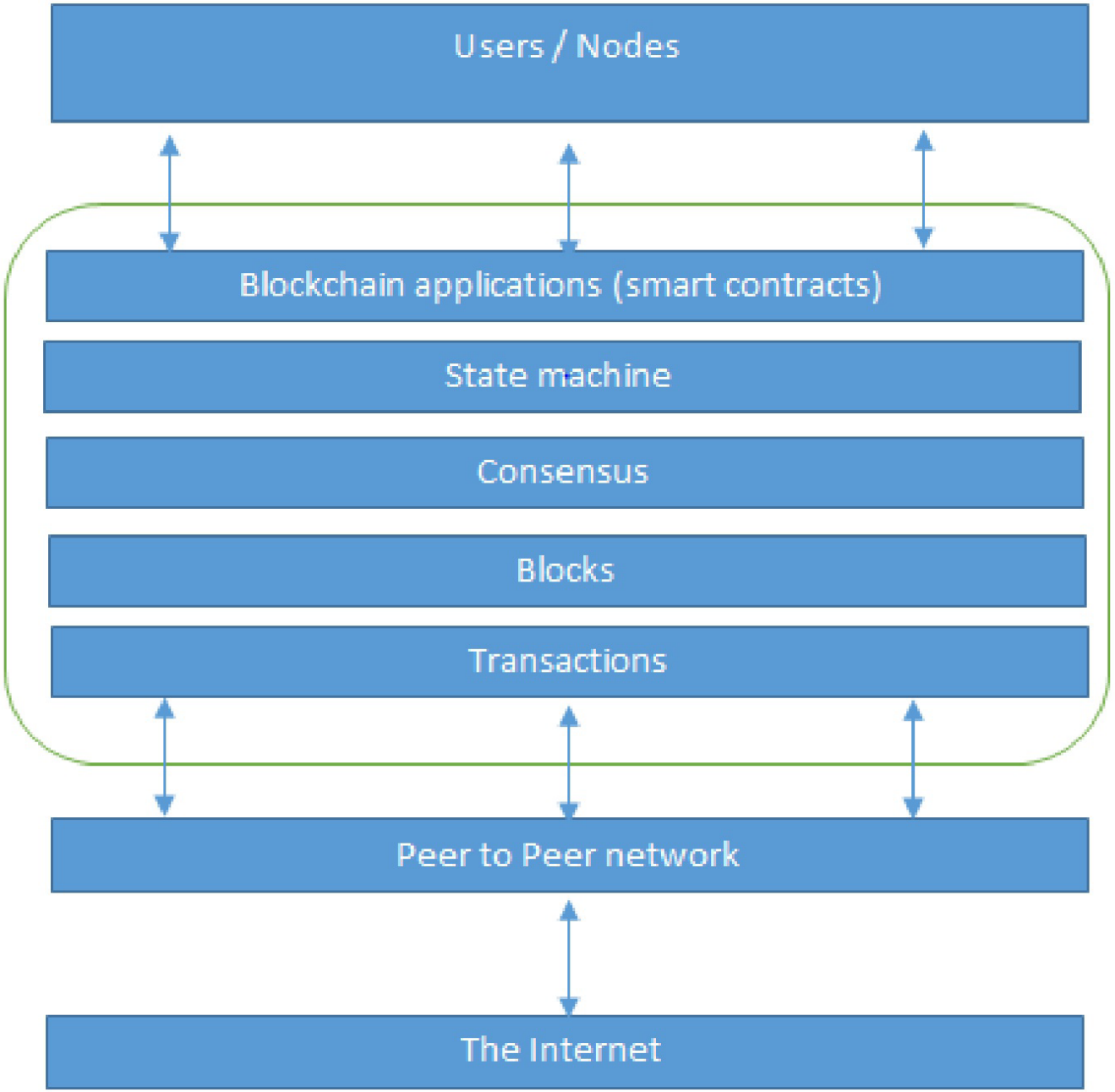
- New data can only be added, not altered or deleted.
- Each block references the previous block (via hash pointer).
- Results in an **immutable chain** of records.

Example: Once a Bitcoin transaction is mined into a block, it’s permanent.

5. Updateable via Consensus

- **Consensus algorithms** ensure all nodes agree on a single version of truth.
- No central control—network collectively validates transactions.
- **Common Consensus Mechanisms:**
  - **Proof of Work (PoW):** Solving cryptographic puzzles (Bitcoin)
  - **Proof of Stake (PoS):** Based on stake ownership
  - **Proof of Authority (PoA):** Selected validators

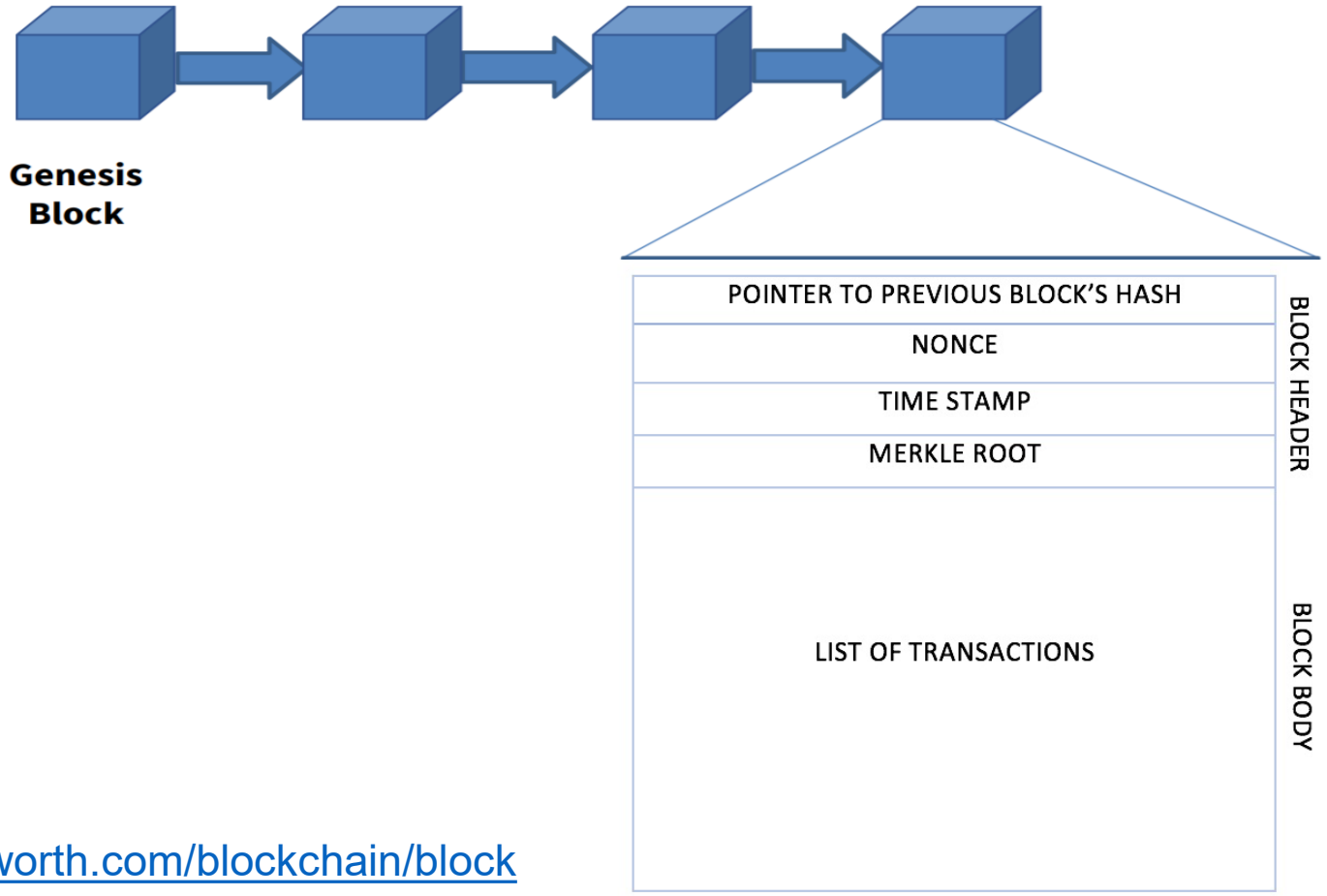
Network Architecture of Blockchain



Component	Description
Transactions	Basic units of data representing value or information exchange
Blocks	Collections of transactions bundled together
Consensus Mechanism	Ensures network agreement on a single version of the ledger
State Machine	Represents the changing global state of the blockchain after each transaction
Smart Contracts	Code executing predefined logic automatically

Structure of a Block

Components of a typical Block in a Distributed Ledger /Blockchain



Demo :- https://andersbrownworth.com/blockchain/block

Structure of a Block

- **A block** is a data structure that contains a set of validated transactions.

Component	Description
Block Header	Contains metadata (previous block hash, timestamp, Merkle root, nonce)
Transactions	List of transactions included in the block
Nonce	A random number used in mining
Merkle Root	A hash summarizing all transactions in the block

Types of Blockchains

Based on Visibility of Network

- **Public**
  - Publicly available: need a client to connect to it
  - Anyone can read, write, and join network
  - Nobody has control over network
  - Open to the public, and anyone can participate as a node in the decision-making process
- **Private**
  - Closed network; participation limited to selected members or organization, deciding who joins and their roles
  - Access restricted to known, authenticated participants

Types of Blockchains

Based on Visibility of Network

- **Semiprivate (Consortium** and **Hybrid)**
  - A bridge between public and private blockchains
  - They provide controlled participation while still allowing a degree of transparency and decentralization
  - **Consortium (Federated) Blockchains**
    - multiple organizations collectively maintain and govern the network.
    - No single entity controls the ledger
    - Only authorized participants can read/write or validate transactions
    - Consensus is achieved among trusted participants.
  - **Hybrid Blockchains**
    - Combines features of both public and private blockchains, allowing selective data visibility and controlled participation.
    - Managed by an organization or consortium.
    - Public users can access limited parts of the blockchain for verification.
    - Private Layer: Sensitive data is stored and accessed by authorized entities.
    - Public Layer: Certain data (e.g., audit trails or proofs) is made public.

Types of Blockchains

Based on Accessibility

- Blockchain network enables their members to participate in the process of block verification.

- **Permissioned**
  - Few members verify data
  - Known participants; regulated access
- **Permission-less**
  - Anyone can join and verify

Types of Blockchains

Based on the presence or absence of a native token (cryptocurrency)

Blockchain network enables their members to participate in the process of block verification.

- **Tokenized**
  - Generates cryptocurrency
  - tokens act as a medium of value transfer between participants
- **Tokenless**
  - does not have a native cryptocurrency or token
  - Its purpose is to provide a secure, shared ledger for data exchange — not value transfer

Types of Blockchains - Variants

Public Permission-less Blockchain

- Well-known implementation in Blockchain
- **Anyone can participate** without permission
- Require Native Assets (Crypto-Currency)
- **Transaction are Transparent**
- Remove middleman in current business system
- Low Scalable
- Example: Bitcoin, Ethereum

Types of Blockchains - Variants

Public Permissioned Blockchain

- The **transactions(ledger) are transparent**.
- Writing, validating, or specific actions **require permission and identity verification**
- Often managed by multiple organizations or a central authority for specific purposes
- Maintainer has the ability to appoint privileged parties
- Only **privileged** parities can **write data**.
- Require native assets
- Example: Ripple, EOS

Types of Blockchains - Variants

Private Permission-less Blockchain

- Only participants with **known identity can join and read data**
- **All** of the participants can **write data**
- Data is confidential
- Does not require native assets
- High Scalability
- Example: LTO Network, Holochain

Types of Blockchains - Variants

Private Permissioned Blockchain

- Enterprise Oriented
- The **enterprise writes and verifies** each transactions.
- Only **privileged parties** can write data.
- Verify transactions internally.
- Reduces transaction costs and data redundancies and replaces legacy systems, simplifying document handling.
- May have security breaches.
- Example Hyperledger Fabric

Types of Blockchains

Type	Description	Examples
Public Blockchain	Open to everyone; permissionless	Bitcoin, Ethereum
Private Blockchain	Controlled access; consortium members only	Quorum, Hyperledger
Semi-Private Blockchain	Partially public, partially private	Hybrid/consortium models
Permissioned Ledger	Known participants; regulated access	Corda, Fabric
Fully Private/Proprietary	Internal organizational use	Govt./Enterprise systems
Tokenized Blockchain	Generates cryptocurrency	Bitcoin, Ethereum
Tokenless Blockchain	No value transfer; for data sharing	Hyperledger Fabric

Consensus Mechanisms in Blockchain

Consensus

- process of agreement among distributed or distrustful nodes on the final state of data in a blockchain network.
- It ensures that all participants agree on a single version of truth — the valid state of the ledger — even if some nodes fail or act maliciously
- Consensus is the backbone of blockchain technology.
- It provides:
  - —Decentralization of control
  - —Security against fraud or tampering
  - —Uniform agreement on valid transactions
  - —Network reliability even with faulty or malicious nodes

Consensus Mechanisms in Blockchain

Consensus and Mining

- Mining is an *optional process* used by some blockchains (e.g., Bitcoin) to achieve consensus through competition.
- Miners solve computational puzzles to propose valid blocks.
- not all blockchains use mining — the choice of consensus mechanism depends on:
  - —Blockchain type (public, private, or permissioned)
  - —Performance, security, and energy considerations

Consensus Mechanism

- A **consensus mechanism** is a **set of rules or steps** that all participating nodes follow to agree on a proposed block or transaction before it's added to the ledger.
- Requirements of a Consensus Mechanism

Requirement	Description
Agreement	All honest nodes must decide on the exact same value.
Termination	All honest nodes eventually reach a decision and finish the process.
Validity	The agreed-upon value must have been proposed by at least one honest node.
Fault Tolerant	The algorithm must function even if nodes are faulty or malicious (Byzantine nodes).
Integrity	No node can make a decision more than once in a single consensus cycle.

General Categories of Consensus

- Broadly, consensus mechanisms fall into two historical categories based on how they handle faults:

A. Traditional Byzantine Fault Tolerance (BFT)-based

- Relies on a system of publisher-signed messages.
- Agreement is reached when a specific threshold of messages is received.
- No compute-intensive operations (unlike mining).

B. Leader Election-based (Lottery-based)

- Nodes compete in a "lottery" to become the leader.
- The winner proposes the final value.

Traditional Consensus Protocols (Non-Blockchain)

Protocol	Description
Paxos (1989)	Uses roles: Proposer, Acceptor, and Learner; consensus achieved by majority agreement.
RAFT	Elects a leader node that coordinates agreement; states: Follower, Candidate, Leader.
PBFT (Practical Byzantine Fault Tolerance)	Handles malicious nodes through message voting and replication.

Consensus in blockchain

- Consensus in blockchain determines how all peers **agree on the valid block** and **maintain the same ledger**.
- Two major types

Type	Category	Description
Proof-based / Leader-election	**Permissionless (Public)**	Used in open networks (Bitcoin, Ethereum). Random leader chosen (e.g., mining).
Voting-based / BFT-based	**Permissioned (Private/Consortium)**	Used in closed systems with known participants (e.g., Hyperledger).

Major Consensus Algorithms in blockchain

Algorithm	Explanation	Used
Proof of Work (PoW)	• Nodes (miners) compete to solve a mathematical puzzle (hashing) • The first to solve it earns the right to add the next block and is rewarded.	Bitcoin, Litecoin, and other cryptocurrency blockchains
Proof of Stake (PoS)	Validators are chosen based on their **stake** (amount of tokens locked in the network)	Ethereum 2.0, Cardano, Algorand
Delegated Proof of Stake (DPoS)	Each node that has a stake in the system can delegate the validation of a transaction to other nodes by voting.	EOS, BitShares, TRON
Proof of Elapsed Time (PoET)	• Each node waits for a random time in a Trusted Execution Environment (TEE) — the one with the shortest wait time wins. • Requires Intel SGX hardware for security.	Hyperledger Sawtooth

Major Consensus Algorithms in blockchain

Algorithm	Explanation	Used
Proof of Deposit (PoD)	• Nodes must **deposit** a certain security amount before participating. • If they behave maliciously, their deposit is **slashed**.	Tendermint
Proof of Importance (PoI)	• Ranks nodes by their **network activity** and **stake**, not just ownership. • Stake + Transaction Volume + Network Reputation	NEM Blockchain
Federated Byzantine Consensus (FBA)	Nodes maintain a list of **trusted peers** and agree based on votes from that subset.	Stellar, Ripple
Reputation-based Consensus	• Nodes build **reputation** through honest behavior. • Leaders are chosen based on this trust score.	GoChain

Major Consensus Algorithms in blockchain

Algorithm	Explanation	Used
PBFT (Practical Byzantine Fault Tolerance)	• Each node exchanges messages and votes until 2/3 majority agreement is reached.	Hyperledger Fabric, Zilliqa
Proof of Activity (PoA)	Combination of **PoW + PoS**: • Miners create a block header (PoW) • Stakeholders sign and confirm (PoS)	Decred (DCR), Espers (ESP)
Proof of Capacity (PoC)	Uses **hard disk space** instead of CPU to mine blocks (“hard drive mining”).	Burstcoin
Proof of Storage / Space-Time	• Nodes prove they are **storing specific data** over time. • Rewards are distributed based on storage contribution.	Filecoin, Chia

Assignment-I

- Tiers of blockchain technology (blockchain tiers: 1.0, 2.0, 3.0, and beyond)
- Explain CAP theorem and how blockchain manages to achieve all of these properties—or does it?
- Differentiate Fully Centralized, Semi Decentralized, Fully Decentralized

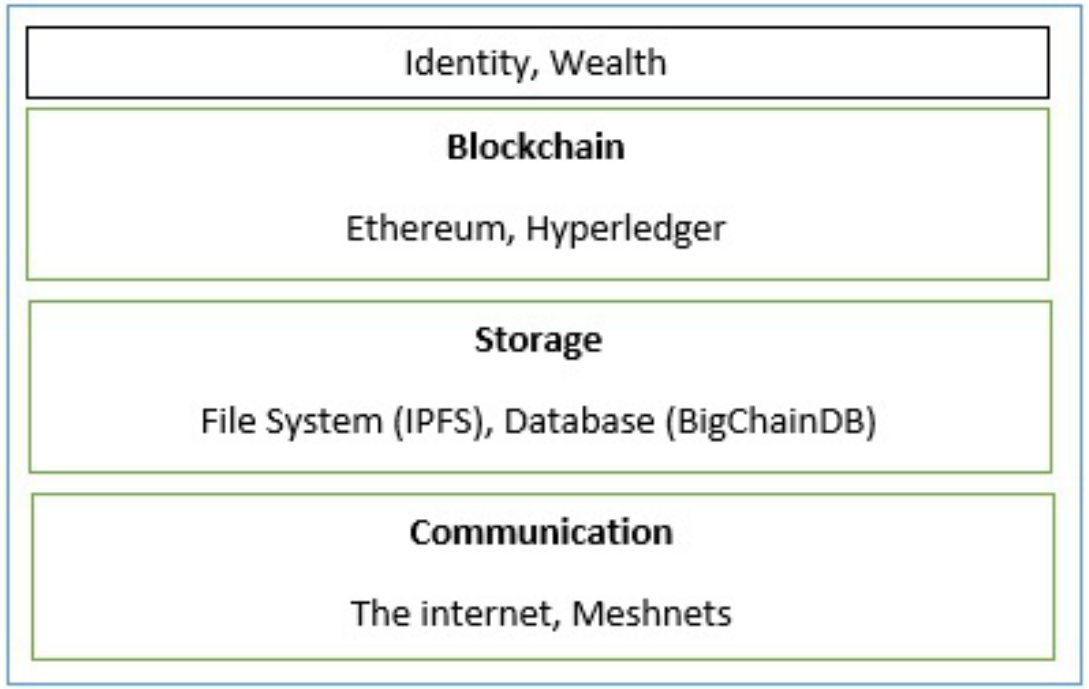
Blockchain and Full Ecosystem Decentralization

- **Decentralization** is the foundational principle of blockchain.
- *True decentralization* requires not only the blockchain itself to be decentralized but also the **entire ecosystem** surrounding it — including **storage**, **communication**, **computation**, **identity**, and **wealth**.
- A blockchain network cannot be truly decentralized if it still depends on centralized servers, cloud storage, or communication systems.

Decentralized Ecosystem

Blockchain as a Layered Structure

- A blockchain operates as a **distributed ledger** running on top of multiple technological layers.
- Each layer — from storage to computation — must contribute to decentralization.
- **Layers in a decentralized ecosystem:**



Decentralized Ecosystem

Storage Layer

- While data *can* be stored directly on a blockchain, it's **not efficient** for large-scale data (like files, images, or videos).
- Blockchains are designed for **transactions**, not for **data storage**.
- **On-chain storage:** Secure but limited and costly.
- **Off-chain decentralized storage:** Needed for scalability.

Decentralized Storage Solutions

Technology	Description
IPFS (InterPlanetary File System)	Uses *Kademlia DHT* + *Merkle DAG* for decentralized file storage and retrieval.
Filecoin	Incentive mechanism for storing data – which pays incentives to nodes that store data
Swarm (Ethereum)	Decentralized storage layer for Ethereum
Storj, MaidSafe	Commercial decentralized cloud storage networks
BigchainDB	Hybrid between blockchain and distributed database - provides blockchain's immutability and ownership control but with database-like speed and scalability

Decentralized Ecosystem

Communication Layer

- **Centralized Internet: The Problem**
- The **modern internet** is dominated by centralized service providers (Google, Amazon, ISPs).
- Even though TCP/IP is decentralized by design, most services now rely on central servers.
- This introduces **points of control and censorship**.
- An alternative is to use **mesh networks** – provide a decentralized alternative where nodes can talk directly to each other without a central hub such as an ISP.

Decentralized Ecosystem

Computing / Processing Layer

- **Decentralized Computing Concept**
- Computation decentralization refers to running programs or logic **across distributed nodes** instead of a single server.
- Blockchain as the Computation Layer

Blockchain Platform	Computation Mechanism	Description
Ethereum	Ethereum Virtual Machine (EVM)	Executes smart contracts across all nodes.
Hyperledger Fabric	Chaincode (containerized code)	Executes logic among verified peers.
Tezos, Solana	Custom virtual machines	Optimized for high throughput and contract execution.

Decentralized Ecosystem

Identity

- Traditional identity systems (like government IDs or login credentials) are centralized.
- A decentralized identity ensures **self-sovereign ownership** of personal information.

Decentralized Identity Technologies:

- **BitAuth, OpenID**
- **Self-Sovereign Identity (SSI) Frameworks**

Wealth

- **Cryptocurrencies** like Bitcoin and Ethereum represent **decentralized wealth systems**.
- Users control their assets directly with **private keys** — no bank or middleman.
- These tokens can represent **value, ownership, or voting rights** in decentralized applications.

Decentralized Organizations and Applications

- Decentralization in blockchain is not limited to data and transactions — it now extends to **organizations, governance, and software systems**.
- Using **smart contracts**, organizations can operate without centralized control – known as **Decentralized Organizations (DOs)**
- When automation is added, they become **Decentralized Autonomous Organizations (DAOs)**.

Decentralized Organizations (DOs)

- A **Decentralized Organization (DO)** is a **software-based organization** that operates on blockchain through smart contracts
- DOs mimic real-world organizations but run via blockchain logic
- Smart contracts replace manual decision-making
- Participants interact directly based on predefined code rules

Decentralized Autonomous Organizations (DAOs)

- A **DAO** is an **autonomous** version of a DO — a **self-governing organization** encoded as smart contracts that execute automatically once certain conditions are met

Aspect	Description
Autonomy	Operates without human intervention once deployed.
Governance	Decisions are made collectively by **token holders** through proposals and voting.
Transparency	All transactions and rules are visible on the blockchain.
Immutability	The DAO's code defines its rules and can only change via consensus.

Module II

Introduction – Bitcoin – Digital Keys and addresses – Transactions – Blockchain – Mining – alternative coins – alternatives to proof of work – Various stake types – Bitcoin limitations – Smart contracts

Introducing Bitcoin

- Bitcoin is the **first and most successful application of blockchain technology**, introduced in **2008** by a pseudonymous creator **Satoshi Nakamoto** through the whitepaper

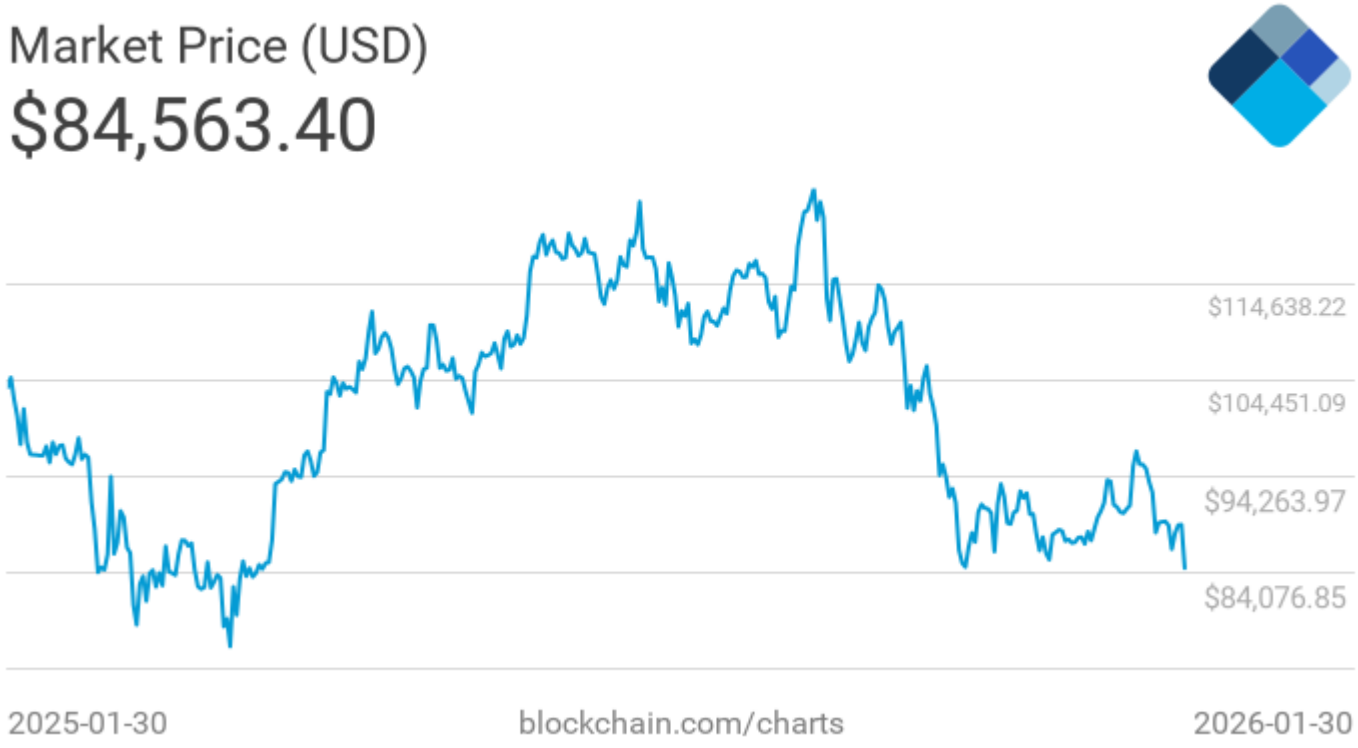
"Bitcoin: A Peer-to-Peer Electronic Cash System"

https://bitcoin.org/bitcoin.pdf

- It is the **first fully decentralized digital currency** that allows **peer-to-peer transactions** without the need for intermediaries such as banks or governments.

Introducing Bitcoin

- As of late January 2026, Bitcoin's market cap stands at approximately **$1.68 trillion**



What is Bitcoin?

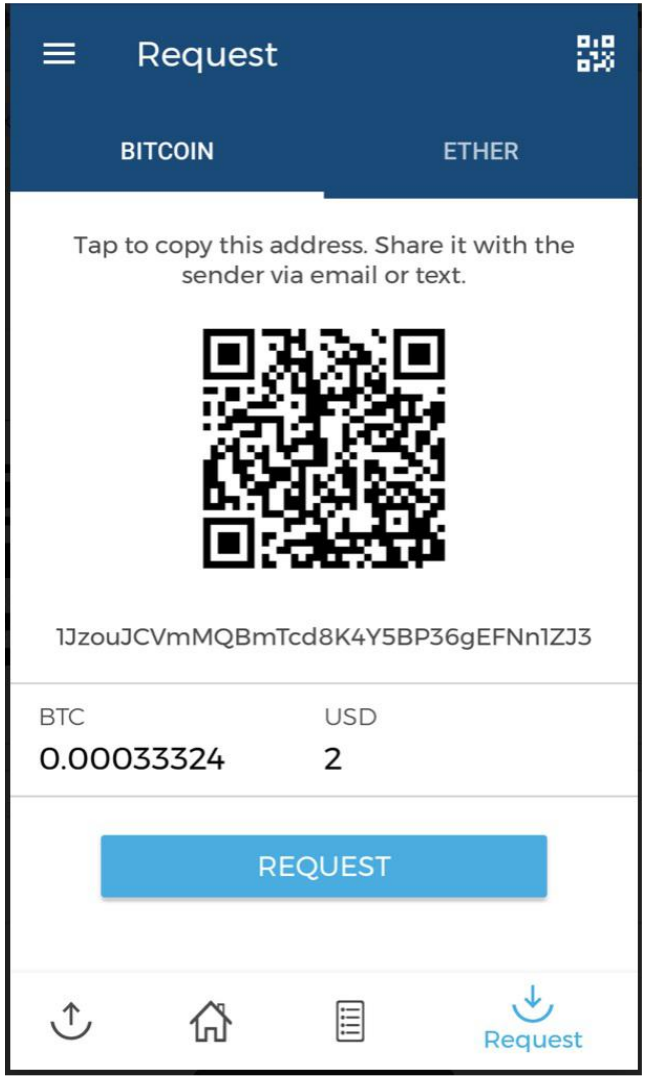
- Three-fold definition: it's a protocol, a digital currency, and a platform
- A combination of peer-to-peer network, protocols, software that facilitate the creation and usage of the digital currency - **Bitcoin**
- Network Effect: The more users join, the more valuable the network becomes.

What is Bitcoin?

- A peer-to-peer electronic cash system that eliminates the "Double-Spending" problem without a central bank.
- Double spending problem
  - a user sends coins to two different users at the same time and they are verified independently as valid transactions.
- The double spending problem is resolved in Bitcoin by using a distributed ledger (blockchain)
  - where every transaction is recorded permanently and by implementing transaction validation and confirmation mechanism.

Sending a payment

- How money can be sent using Bitcoin network from one user to another?
- Using Blockchain wallet for mobile devices:
  1. The payment is requested from a user by sending the Bitcoin address



Sending a payment

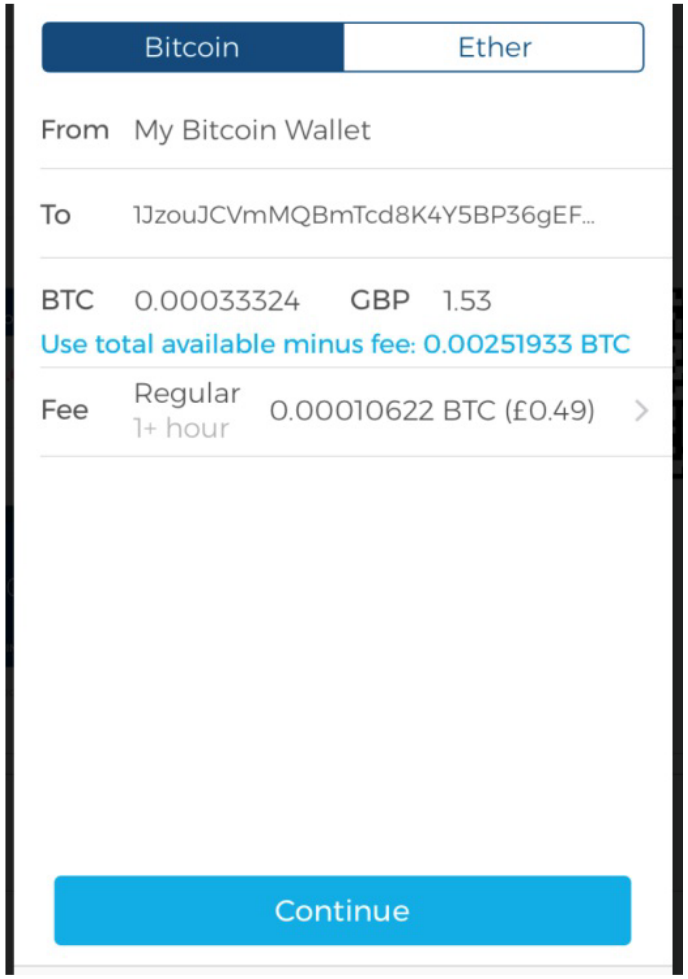
- How money can be sent using Bitcoin network from one user to another?
- Using Blockchain wallet for mobile devices:
  1. The payment is requested from a user by sending the Bitcoin address
  2. The sender either enters the receiver's address or scans the QR code that has the Bitcoin address

Please send 0.00033324 BTC to the Bitcoin address 1JzouJCVmMQBmTcd8K4Y5BP36gEFNn1ZJ3.

bitcoin://1JzouJCVmMQBmTcd8K4Y5BP36gEFNn1ZJ3?amount=0.00033324

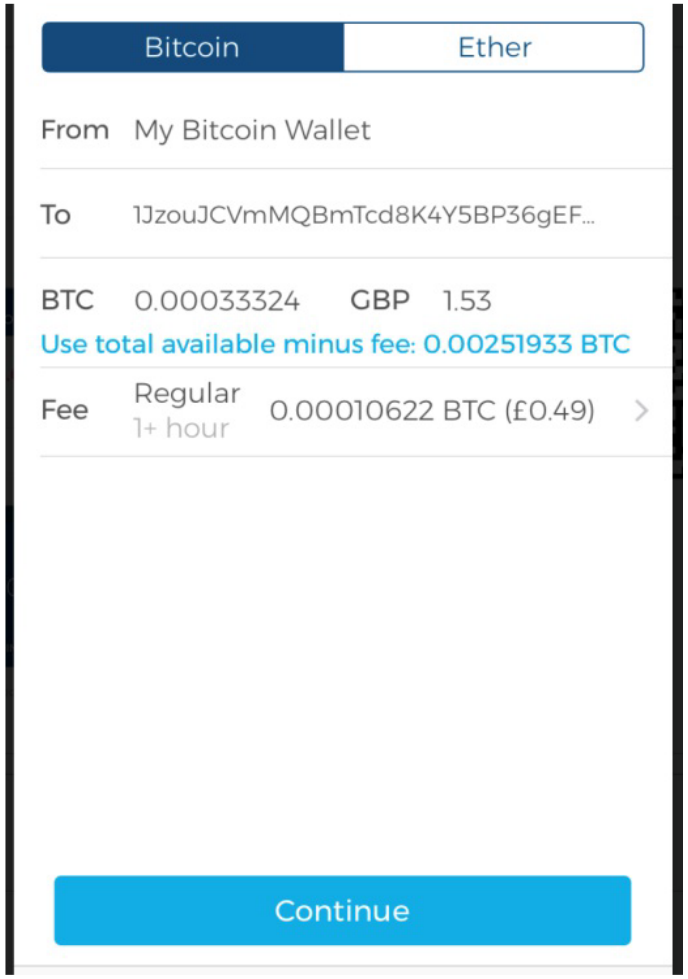
Sending a payment

- How money can be sent using Bitcoin network from one user to another?
- Using Blockchain wallet for mobile devices:
  1. The payment is requested from a user by sending the Bitcoin address
  2. The sender either enters the receiver's address or scans the QR code that has the Bitcoin address
  3. In the wallet application of the sender, this transaction is constructed and broadcasted to the Bitcoin network. This transaction is digitally signed using the private key of the sender before broadcasting it.



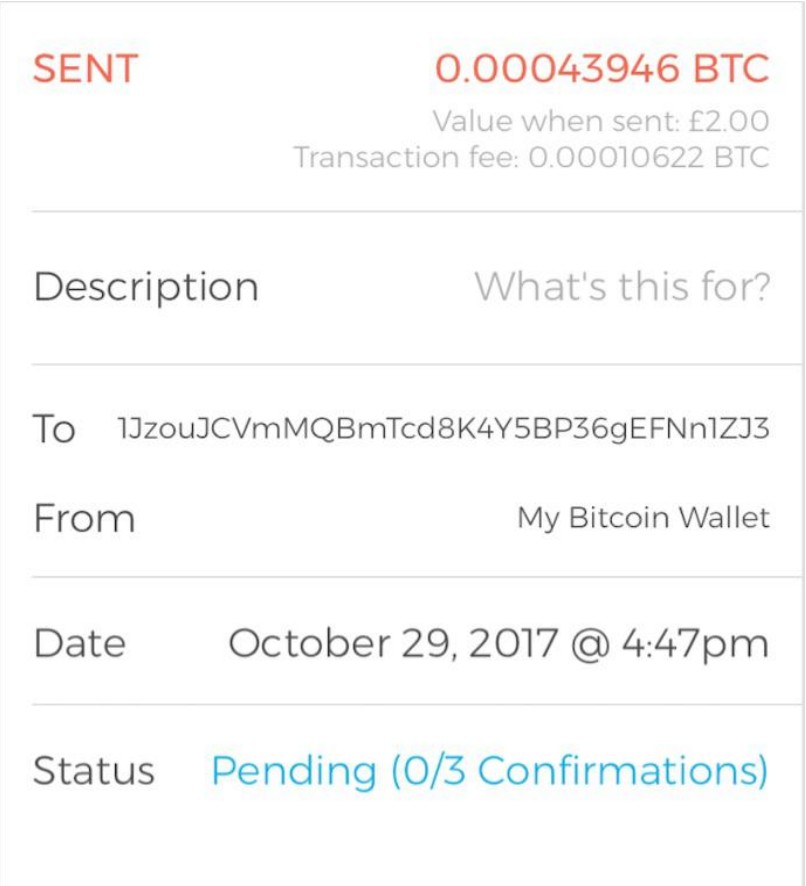
Sending a payment

- Fee is calculated based on the size of the transaction and a fee rate is a value that depends on the volume of the transaction in the network
- Fee in Bitcoin network ensures that your transaction will be included by miners in the block.



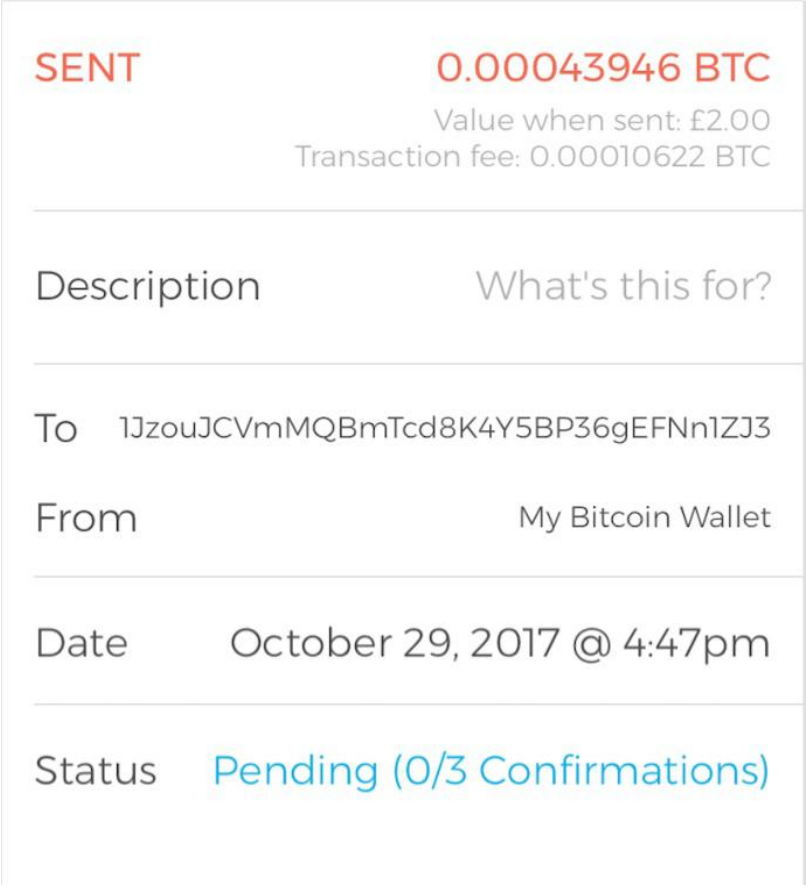
Sending a payment

- How money can be sent using Bitcoin network from one user to another?
- Using Blockchain wallet for mobile devices:
  1. The payment is requested from a user by sending the Bitcoin address
  2. The sender either enters the receiver's address or scans the QR code that has the Bitcoin address
  3. In the wallet application of the sender, this transaction is constructed and broadcasted to the Bitcoin network. This transaction is digitally signed using the private key of the sender before broadcasting it.
  4. The transaction has been constructed, signed and sent out to the Bitcoin network.



Sending a payment

- Confirmation is pending for this transaction
- These confirmations will start to appear as soon as the transaction is verified, included in the block, and mined
- The appropriate fee will be deducted from the original value to be transferred and will be paid to the miner who has included it in the block for mining



1PL6gsm49xCFMvrXqgGcee5cdrG119GoWN (0.00137322 BTC - Output) → 1JzouJCVmMQBmTcd8K4Y5BP36gEFNn1ZJ3 - (Unspent) 0.00033324 BTC
1ET3oBGf8JpunjytE7owyVtmBjmvcDycQe - (Unspent) 0.00093376 BTC

0.001267 BTC

Summary	
Size	226 (bytes)
Weight	904
Received Time	2017-10-29 16:47:58
Included In Blocks	492229 (2017-10-29 16:51:42 + 4 minutes)
Confirmations	731 Confirmations
Visualize	View Tree Chart

Inputs and Outputs	
Total Input	0.00137322 BTC
Total Output	0.001267 BTC
Fees	0.00010622 BTC
Fee per byte	47 sat/B
Fee per weight unit	11.75 sat/WU
Estimated BTC Transacted	0.00033324 BTC
Scripts	Hide scripts & coinbase

- Size: Size of the transaction in bytes.
- Weight: Metric for block and transaction size since the introduction of Segregated Witness (SegWit) version of Bitcoin.
- Received Time: Time when the transaction is received.
- Included In Blocks: Block number on the blockchain in which the transaction is included.
- Confirmations: Number of confirmations by miners for this transaction.

- Total Input: Number of total inputs in the transaction.
- Total Output: Number of total outputs in the transaction.
- Fees: Total fees charged.
- Fee per byte: The total fee divided by the number of bytes in a transaction
- Fee per weight unit: calculated using *total number of bytes * 4*

Bitcoin Transactions

- Current transaction on the Bitcoin network

  Blockchain Explorer - Bitcoin Tracker & More | Blockchain.com

The payment transaction in the Bitcoin network can be divided into the following steps:

1. Transaction starts with a sender signing the transaction with their private key
2. Transaction is serialized so that it can be transmitted over the network
3. Transaction is broadcasted to the network
4. Miners listening for the transactions picks up the transaction
5. Transaction are verified for their validity by the miners
6. Transaction are added to the candidate/proposed block for mining
7. Once mined, the result is broadcasted to all nodes on the Bitcoin network

Technical Building Blocks

- The main components of a Bitcoin network are:
  - Digital keys
  - Addresses
  - Transactions
  - Blockchain
  - Miners
  - The Bitcoin network
  - Wallets (client software)

Digital Keys and Addresses

- Bitcoin uses **Elliptic Curve Cryptography (ECC)** is used to generate public and private key pairs in the Bitcoin network.

Private Key

- A **256-bit random number**.
- Used to digitally sign the transactions proving the ownership of the bitcoins.
- Must be kept secret.
- Encoded in **WIF (Wallet Import Format)** in order to make them easier to copy and use.

  Private key: A3ED7EC8A03667180D01FB4251A546C2B9F2FE33507C68B7D9D4E1FA5714195201
  WIF format: L2iN7umV7kbr6LuCmgM27rBnptGbDVc8g4ZBm6EbgTPQXnj1RCZP

Public Key

- A public key is 256-bits in length, derived from private key using ECC
- Used by others to **verify signatures**.

Bitcoin Addresses

- Created from public key using:

  RIPEMD-160(SHA-256(public key)) => 160-bit hash

- Hash is then prefixed with a version number and finally encoded with a Base58Check encoding scheme **->** (Version + PubKeyHash + Checksum) is converted into **Base58**
- Common prefixes:

Address Type	Starts With	Example
Legacy (P2PKH)	1	1PL6gsm49xCFMvrXqgGcee5cdrG119GoWN
Pay-to-Script-Hash (P2SH)	3	3KN5tWmLwASxyMSr2wkbzSTxkQLSfkaGka
Native SegWit (Bech32)	bc1q	bc1qrdls5j4zjjhyuxn8qe22pgmn9d8q84za7zy0h9
Taproot (Bech32m)	bc1p	bc1pn5nzthsd2cr7lfu2nmn0ndpg39r5smzshpdsv4jtq6cfxmgadfasr485hl

Bitcoin Transactions

- A **transaction** transfers bitcoin ownership between addresses
- Transactions are not encrypted and are publicly visible in the blockchain

Transaction Life Cycle

1. Sender's wallet creates and signs transaction
2. Broadcast to the network – Flooding Algorithm
3. Verified and stored in **mempool** (transaction pool)
4. Miners select transactions to include in the block.
5. When block mined (solves the PoW) → transaction confirmed
6. Receiver sees confirmation (typically 3–6 confirmations for finality)

Transaction Fee in Bitcoin

- In the Bitcoin network, **transaction fees** are small amounts of Bitcoin paid by the sender to incentivize **miners** for including their transactions in a block
- These fees ensure **network efficiency, security, and prioritization** of transactions.

Purpose of Transaction Fees

- **Incentive for Miners:**
  - Miners spend computational power and electricity to validate transactions.
  - Transaction fees act as a **reward** in addition to the block subsidy (newly created bitcoins).
- **Prevention of Network Spam:**
  - By requiring a fee, the network discourages users from flooding it with meaningless transactions.
- **Prioritization:**
  - Transactions with **higher fees** are **confirmed faster**, as miners select them first from the mempool.

Mempool and Miner Selection

- All **unconfirmed transactions** wait in a **memory pool (mempool)**.
- Miners **select transactions** based on:
  - Fee per byte (sat/byte)
  - Transaction validity
  - Available block space (~1 MB)
- **Higher fee = higher chance of inclusion in the next block.**

Fee Flexibility

- **Not fixed** by the Bitcoin protocol.
- Technically, a transaction **can be sent with zero fee**, but:
  - It may remain **unconfirmed for hours or days**.
  - Nodes may eventually drop it if it stays unconfirmed too long.

Transaction data structure

- Transactions are combined to create a block.

Transaction Components and Scripts

- Each transaction consists of several core components:
  - **Metadata**
  - **Inputs**
  - **Outputs**
  - **Verification** using the **Script language**

```
{
     "lock_time":0,
     "size":226,
     "inputs":[
          {
          "prev_out":{
          "index":139,
          "hash":"40120e43f00ff96e098a9173f14f1371655b3478bc0a558d6dc17a4ab176387d"
          },
          "script":"483045022100de6fd8120d9f142a82d5da9389e271caa3a757b01757c8e4fa7afbf92e74257c02202a78d4fbd52ae9f3a0083
          760d76f84643cf8ab80f5ef971}
     ],
     "version":1,
     "vin_sz":1,
     "hash":"d28ca5a59b2239864eac1c96d3fd1c23b747f0ded8f5af0161bae8a616b56a1d",
     "vout_sz":2,
     "out":[
          {
               "script_string":"OP_DUP OP_HASH160 c568ffeb46c6a9362e44a5a49deaa6eab05a619a OP_EQUALVERIFY OP_CHECKSIG",
               "address":"1JzouJCVmMQBmTcd8K4Y5BP36gEFNn1ZJ3",
               "value":33324,
               "script":"76a914c568ffeb46c6a9362e44a5a49deaa6eab05a619a88ac"
          },
          {
               "script_string":"OP_DUP OP_HASH160 9386c8c880488e80a6ce8f186f788f3585f74aee OP_EQUALVERIFY OP_CHECKSIG",
               "address":"1ET3oBGf8JpunjytE7owyVtmBjmvcDycQe",
               "value":93376,
               "script":"76a9149386c8c880488e80a6ce8f186f788f3585f74aee88ac"
          }
     ]
}
```

Transaction Components and Scripts

Metadata

- Metadata refers to the **basic transaction information** that describes the structure and properties of a Bitcoin transaction.

Field	Size	Description
Version	4 bytes	Specifies the transaction format and rules to be used by nodes and miners.
Size	Variable	Total size of the transaction in bytes. It depends on the number of inputs, outputs, and script lengths.
Input Counter	1–9 bytes	Number of inputs included in the transaction.
Output Counter	1–9 bytes	Number of outputs included in the transaction.
Transaction Hash	32 bytes	Unique identifier (SHA-256 hash) of the entire transaction.

Transaction Components and Scripts

Inputs

- **Role of Inputs**
  - Each input **spends** a previously **unspent output (UTXO)**.
  - UTXO stands for **Unspent Transaction Output** — coins that are available to be spent in future transactions.
  - Once an input uses a UTXO, it becomes **spent** and cannot be reused.

Field	Size	Description
Transaction Hash	32 bytes	The hash of the **previous transaction** containing the UTXO.
Output Index	4 bytes	Identifies which output from the previous transaction is being spent.
Script Length	1–9 bytes	Specifies the size of the unlocking script.
Unlocking Script (ScriptSig)	Variable	Script that **proves ownership** of the bitcoins being spent.
Sequence Number	4 bytes	Used for transaction replacement or lock timing. Disabled by default (`0xFFFFFFFF`).

Transaction Components and Scripts

Outputs

- **Role of Outputs**
  - Outputs define **where the bitcoins go**.
  - Each output includes:
    - The amount being transferred (in **Satoshis**).
    - A **locking script** (ScriptPubKey) that defines how the coins can be spent later.

Field	Size	Description
Value	8 bytes	Number of satoshis (1 BTC = 100,000,000 satoshis).
Script Size	1–9 bytes	Length of the locking script.
Locking Script (ScriptPubKey)	Variable	Script containing **conditions** to unlock and spend the output.

Verification Process

- Verification ensures that:
  - The transaction is **authentic**.
  - The spender **owns** the inputs being used.
  - The transaction **follows** the rules set by the Bitcoin protocol.
- This process is performed using the **Bitcoin scripting language**.

Verification Process – Using Script Language

- Bitcoin uses a **simple, stack-based** scripting language called **Script**.
- It is **not Turing-complete**, meaning it cannot create loops or infinite processes — this ensures safety.
- It uses **Reverse Polish Notation (RPN)** where operands come before operators.
- The evaluation follows a **LIFO (Last In, First Out)** stack method.
- Process to be spent a transaction is evaluated as:
  1. First, it is unlocked and then it is spent
  2. ScriptSig is provided by the user who wishes to unlock the transaction
  3. ScriptPubkey is part of the transaction output and specifies the conditions that need to be fulfilled in order to spend the output
  4. Outputs are locked by ScriptPubKey that contains the conditions, when met will unlock the output, and coins can then be redeemed

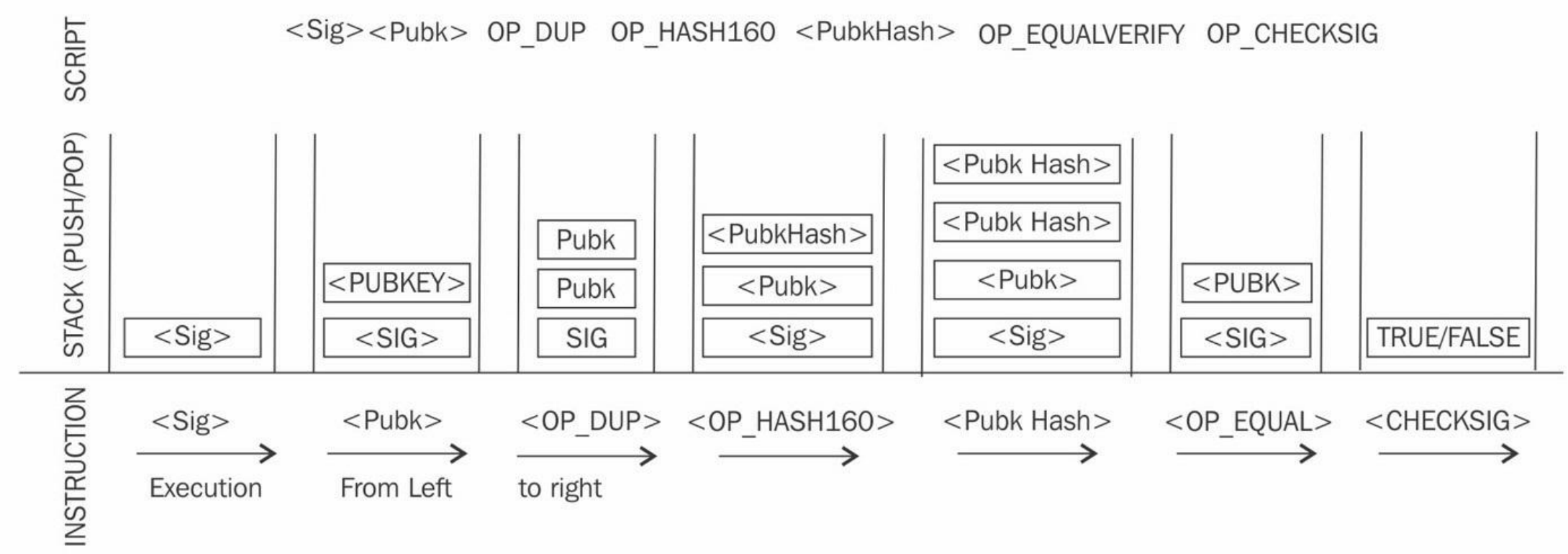
Commonly used Opcodes

Opcode	Description
`OP_CHECKSIG`	Verifies that the digital signature matches the public key.
`OP_EQUAL`	Returns 1 if inputs are equal, else 0.
`OP_DUP`	Duplicates the top stack item.
`OP_HASH160`	Hashes the input using SHA-256 followed by RIPEMD-160.
`OP_VERIFY`	Marks the transaction invalid if the top stack value is not true.
`OP_EQUALVERIFY`	Combines `OP_EQUAL` and `OP_VERIFY` for equality check and validation.
`OP_CHECKMULTISIG`	Validates multiple signatures against multiple public keys (used in multisig).

Types of Bitcoin Transactions

Transaction Type	Description / Template
Pay to Public Key Hash (P2PKH)	Standard type used to send BTC to a wallet address. • ScriptSig: `<sig> <pubKey>` • ScriptPubKey: `OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`
Pay to Script Hash (P2SH)	Sends to a **script hash** (addresses starting with “3”). • ScriptSig: `[<sig>...<sig>] <redeemScript>` • ScriptPubKey: `OP_HASH160 <redeemScriptHash> OP_EQUAL`
MultiSig (Pay to MultiSig)	Requires **M-of-N** valid signatures to redeem. • ScriptSig: `0 [<sig>...<sig>]` • ScriptPubKey: `<m> <pubKey> [...] <n> OP_CHECKMULTISIG`
Pay to Pubkey (P2PK)	Older format; used mainly for coinbase transactions. • Script: `<PubKey> OP_CHECKSIG`
Null Data (OP_RETURN)	Used to store small pieces of data (up to 40 bytes) on the blockchain. • Script: `OP_RETURN <data>` (unspendable output)

P2PKH Script Execution



Coinbase Transactions

- The **first transaction** in every block, created by the **miner**.
- Used to **generate new bitcoins** as a **block reward**.

❖ **Key Properties**

- Has **no input references** (transaction hash set to all zeros).
- Can include up to **100 bytes of arbitrary data** (e.g., messages or tags).
- Creates new coins + collects transaction fees.

Coinbase Transaction Structure

Field	Size	Description
Transaction Hash	32 bytes	All zeros (no previous reference).
Output Index	4 bytes	Always `0xFFFFFFFF`.
Coinbase Data Length	1–9 bytes	Size of the arbitrary data.
Data	Variable	Arbitrary data or message.
Sequence Number	4 bytes	`0xFFFFFFFF`.

The Structure of a Block in Bitcoin

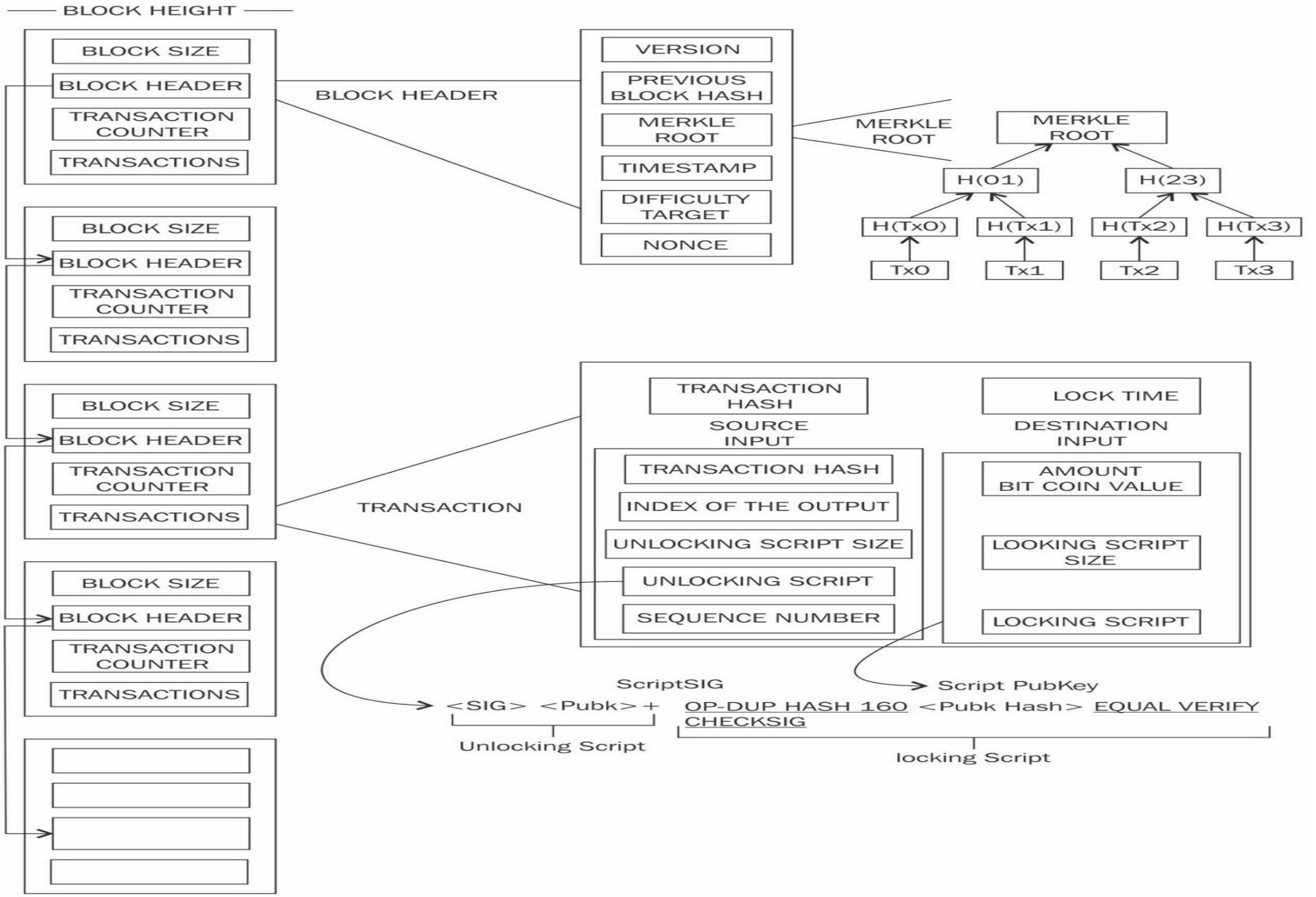
- A **block** is the fundamental unit of data in the **Bitcoin blockchain**.
- It stores a **group of verified transactions** and some **metadata** that ensures the block's validity and linkage to previous blocks.

Structure of a Block

Field	Size	Description
Block Size	4 bytes	Total size of the block (in bytes), indicating how much data it contains.
Block Header	80 bytes	Contains the summary information of the block, explained in the next section.
Transaction Counter	Variable (1–9 bytes)	Number of transactions included in the block, including the **coinbase transaction** (the miner's reward).
Transactions	Variable	The actual list of transactions stored inside the block.

Structure of a Block Header

Field	Size	Description
Version	4 bytes	Indicates which **block validation rules** are used (set by the Bitcoin protocol).
Previous Block's Header Hash	32 bytes	Double SHA-256 hash of the **previous block's header**, linking this block to the previous one.
Merkle Root Hash	32 bytes	Double SHA-256 hash of the **Merkle tree** that summarizes all transactions in the block.
Timestamp	4 bytes	The time (in Unix epoch format) when the miner **started hashing** this block header.
Difficulty Target	4 bytes	Defines how hard it is to find a valid hash (controls mining difficulty).
Nonce	4 bytes	A random number that miners keep changing to find a hash **below the difficulty target**.


BLOCK HEIGHT
BLOCK SIZE
BLOCK HEADER
TRANSACTION COUNTER
TRANSACTIONS
BLOCK HEADER
VERSION
PREVIOUS BLOCK HASH
MERKLE ROOT
TIMESTAMP
DIFFICULTY TARGET
NONCE
MERKLE ROOT
MERKLE ROOT
H(01)
H(23)
H(Tx0)
H(Tx1)
H(Tx2)
H(Tx3)
Tx0
Tx1
Tx2
Tx3
TRANSACTION
TRANSACTION HASH
SOURCE INPUT
TRANSACTION HASH
INDEX OF THE OUTPUT
UNLOCKING SCRIPT SIZE
UNLOCKING SCRIPT
SEQUENCE NUMBER
LOCK TIME
DESTINATION INPUT
AMOUNT BIT COIN VALUE
LOOKING SCRIPT SIZE
LOCKING SCRIPT
ScriptSIG
Script PubKey
<SIG> <Pubk> +
OP-DUP HASH 160 <Pubk Hash> EQUAL VERIFY CHECKSIG
Unlocking Script
locking Script

Mining in Bitcoin

- **Mining** is the process by which **new blocks** are added to the **Bitcoin blockchain**.
- It ensures **network security**, **validates transactions**, and **controls the supply** of bitcoins.
- Miners compete to solve a **mathematical puzzle** (Proof of Work – PoW).
- The winner gets to **add a new block** and is **rewarded** with new bitcoins + transaction fees.

Task of a Bitcoin Miner

Step	Task	Description
1	**Sync with Network**	Download the existing blockchain history and connect to other nodes.
2	**Validate Transactions**	Verify digital signatures and ensure no double spending.
3	**Validate Blocks**	Check correctness of transactions and nonce values in received blocks.
4	**Create a New Block**	Select valid transactions and construct a new candidate block.
5	**Perform Proof of Work**	Find a nonce that produces a hash lower than the difficulty target.
6	**Broadcast and Get Reward**	Announce the new block and receive the block reward + fees if accepted.

Bitcoin Mining Parameters

Parameter	Value / Description
Average Block Time	~10 minutes per block
Current Block Reward (as of 2024)	3.125 BTC (previously 6.25 BTC before halving)
Blocks per Day	~144
Bitcoins per Day	144 × 6.25 = **900 BTC/day**
Total Supply Limit	**21 million BTC** (to be reached ~year 2140)
Reward Halving	Every **210,000 blocks (~4 years)**

Year	Reward per Block	Remarks
2009	50 BTC	Initial reward
2012	25 BTC	1st halving
2016	12.5 BTC	2nd halving
2020	6.25 BTC	3rd halving
2024	3.125 BTC	4th halving

Proof of Work (PoW)

- PoW ensures miners have **spent computational effort** to secure the network.
- **Formula:**

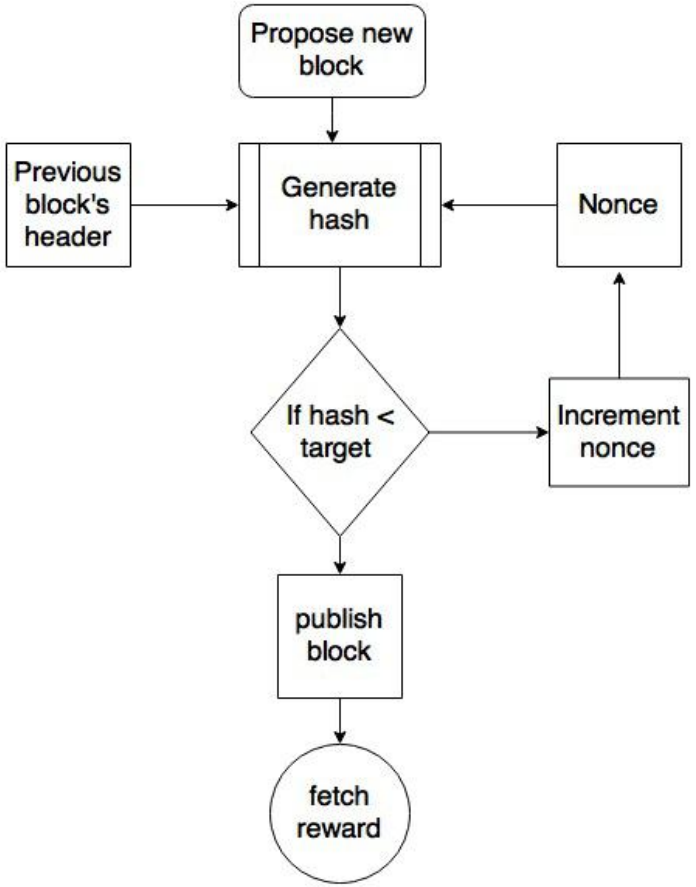
$$H(N \mid\mid P_{hash} \mid\mid Tx_1 \mid\mid Tx_2 \mid\mid \ldots \mid\mid Tx_n) < Target$$

Where:

- **H** = Double SHA-256 hash function
- **N** = Nonce (variable number miners change)
- **P_hash** = Previous block hash
- **Tx_1 … Tx_n** = Transactions in the block
- **Target** = Difficulty threshold set by the network

The mining algorithm

1. Retrieve **previous block's header** from the network.
2. Assemble valid transactions into a **new block**.
3. Compute: Hash=SHA256(SHA256(previous block's header + nonce +new block))
4. Check: If Hash < Difficulty Target → Block Found!
5. If not, **increment the nonce** and **retry**.
6. Once a block is found, broadcast it and **collect the reward**.



Alternative Coins (Altcoins)

- After the success of **Bitcoin (2009)**, many new **alternative cryptocurrency projects** were launched.
- These are collectively known as **Altcoins** (Alternative Coins).
- **First altcoin:** Namecoin (2011)
  - Major growth period: **2013–2014**
  - Current market: Hundreds of altcoins, many with monetary value
  - Some altcoins succeeded (e.g., **Zcash**, **Litecoin**), while others failed or turned out to be scams (“pump and dump” schemes).

Purpose of Altcoins

- Altcoins often aim to:
  - Solve Bitcoin's **limitations** (e.g., scalability, privacy, block time).
  - Experiment with **new consensus algorithms**.
  - Provide **specialized applications** like decentralized identity, naming, or data storage.
- Example:
  - **Namecoin:** Decentralized domain name and identity system (not mainly for payments).
  - **Zcash:** Enhanced privacy using zero-knowledge proofs.
  - **Litecoin:** Faster block generation using Scrypt algorithm.

Module III

Ethereum – Introduction – Ethereum network – Components of Ethereum ecosystem – EVM – Development tools and Frame works – Remix – Truffle – Solidity language

Ethereum

- **Ethereum** is a decentralized, open-source blockchain platform that extends Bitcoin's functionality by supporting **smart contracts**.
- It was **conceptualized by Vitalik Buterin in November 2013** and officially launched in **2015**.
- The critical idea proposed was
  - the development of a Turing-complete language that allows the **development of arbitrary programs (smart contracts)** for blockchain and decentralized applications.

Ethereum

- **Ethereum Yellow Paper**
  - Written by **Dr. Gavin Wood**, co-founder of Ethereum and Parity Technologies.
  - It provides a **formal mathematical specification** of Ethereum protocol.
  - Paper Title : ‘Ethereum: A Next-Generation Cryptocurrency and Decentralized Application Platform’
  - Document URL: https://ethereum.github.io/yellowpaper/paper.pdf

Ethereum Release History & Roadmap

Proof of Work (PoW) | PoS Phase

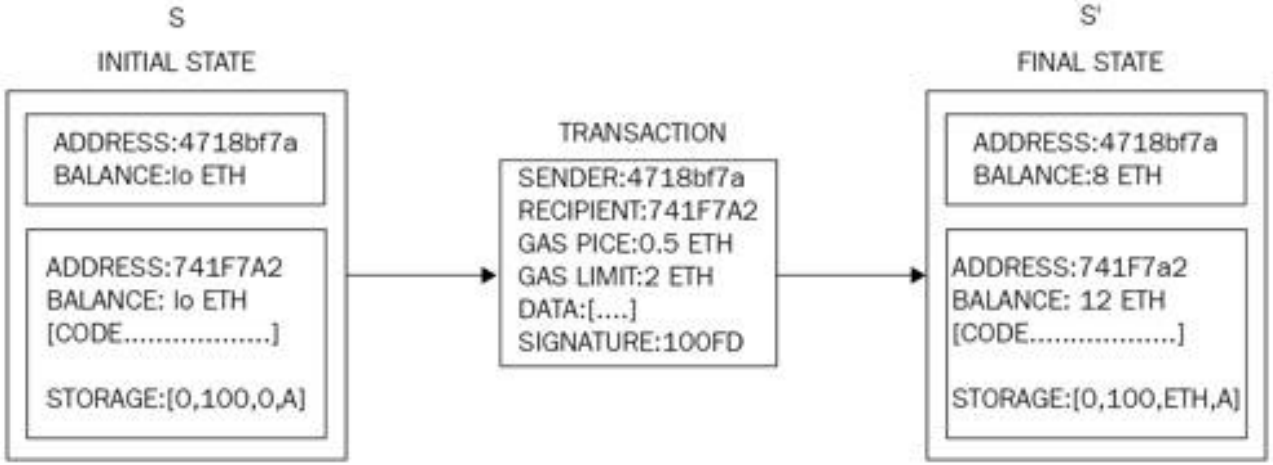
May 2015	30 July 2015	14 March 2016	16 October 2017	28 February 2019	8 December 2019	5 August 2021	December 2025
Olympic (Testnet) Version 0	Frontier (Beta) Version 1	Homestead Version 2	Metropolis (v. Byzantium [1]) Version 3	Metropolis / (St. Petersburg) Version 3.5	Istanbul Version 4	London EIP-1559 Fork	Future

Ethereum blockchain

- Ethereum is described as a **transaction-based state machine**:

  Each block executes transactions that transform the global state (world state) into a new valid state.

- For example:
  - If **Address A** sends **2 Ether** to **Address B**,
    → Ethereum's global state is updated to reflect this new balance distribution.



The core idea in Ethereum blockchain,

- a genesis state is transformed into a final state by executing transactions incrementally.
- the final transformation is then accepted as the absolute undisputed version of the state.

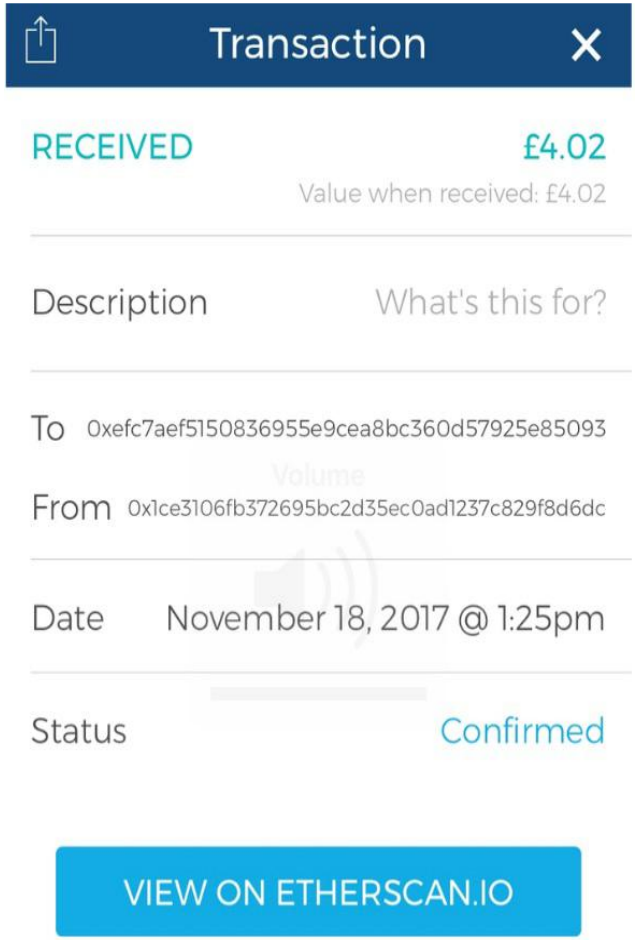
Ethereum from a User's Perspective

- Let's consider a **simple Ether transfer** between two users:

Step	Description
1	Receiver (e.g., Bob) sends Ethereum address to sender (Alice).
2	Alice scans the QR code or copies the address in wallet (e.g., **Jaxx Wallet**).
3	Transaction is created and digitally **signed** by Alice's **private key**.
4	The signed transaction is **broadcast** to the Ethereum network.
5	**Miners** pick it up, verify it, and include it in a block.
6	A miner solves the **Proof of Work (PoW)** using a valid nonce.
7	The block is added to the chain, and **Bob receives Ether**.

Tap to copy this address. Share it with the sender via email or text.

0xeFc7aEF5150836955e9CEa8Bc360D57925e850...



On the blockchain, this transaction is identified by the following transaction hash:

`0xc63dce6747e1640abd63ee63027c3352aed8cdb92b6a02ae25225666e171009e`

TxHash:
0xc63dce6747e1640abd63ee63027c3352aed8cdb92b6a02ae25225666e171009e

TxReceipt Status:
Success

Block Height:
4576084 (20583 block confirmations)

TimeStamp:
3 days 7 hrs ago (Nov-18-2017 01:25:54 PM +UTC)

From:
0x1ce3106fb372695bc2d35ec0ad1237c829f8d6dc

To:
0xefc7aef5150836955e9cea8bc360d57925e85093

Value:
0.015927244142974896 Ether ($5.82)

Gas Limit:
21000

Gas Used By Txn:
21000

Gas Price:
0.000000021 Ether (21 Gwei)

Actual Tx Cost/Fee:
0.000441 Ether ($0.16)

Cumulative Gas Used:
156148

Nonce:
1

can view Ethereum transactions on Etherscan.io.

The Ethereum network

- Ethereum operates as a **peer-to-peer (P2P)** network of nodes.
- There are three main types of networks:

Network	Purpose	Chain ID
Mainnet	Live Ethereum blockchain used for real ETH transactions	1
Testnet	Used for testing smart contracts without real value	its own distinct ID
Private Net	Custom network for organizations or developers	Custom

The current version of the Ethereum mainnet operates under a Proof-of-Stake (PoS) consensus mechanism

Ethereum Mainnet

- Ethereum Mainnet is the **primary, live blockchain network** where **actual ETH** transactions, **smart contracts**, and decentralized applications **(DApps)** operate with real-world value.
- It is the most secure, immutable, and decentralized version of the network, using **Chain ID 1** for all transactions.
- It differs from testnets by using **real funds for gas fees**.

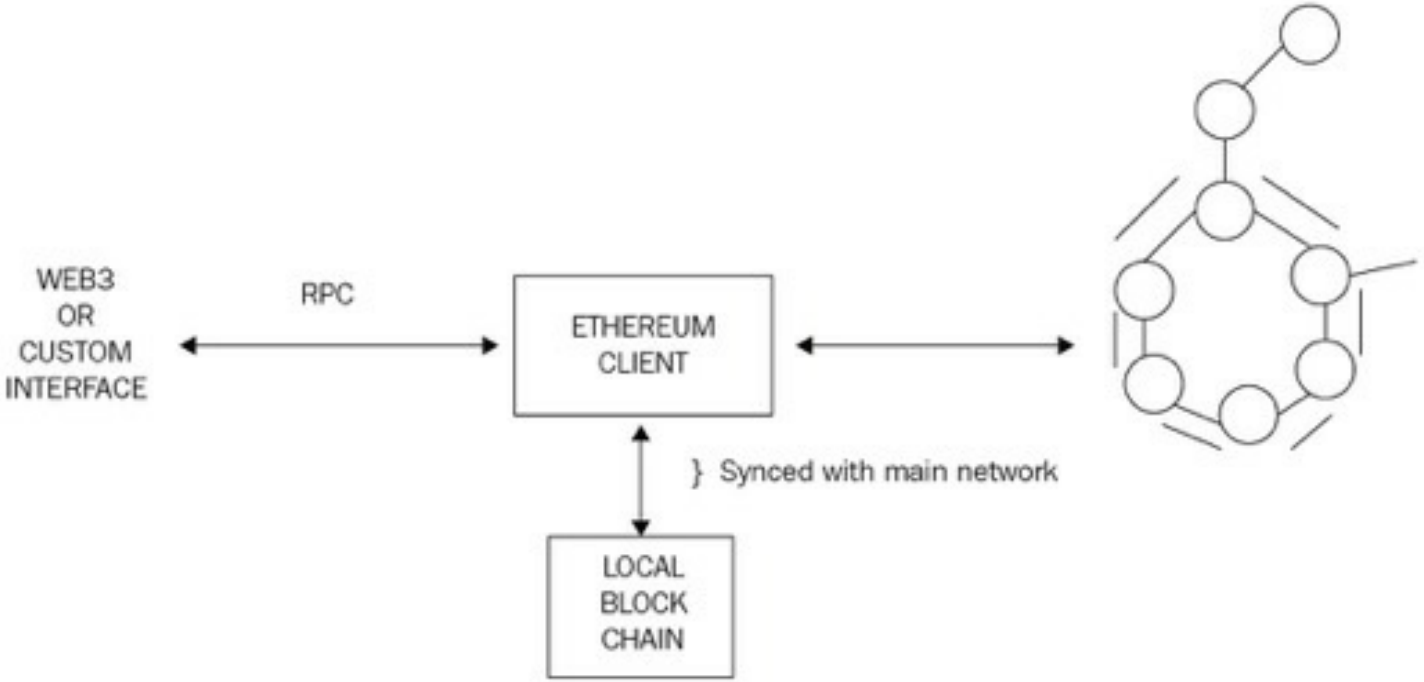
Testnet

Ethereum uses several public testnets with unique Chain IDs for developers to test smart contracts and protocol upgrades before deploying to the Mainnet.

Network Name	Chain ID / Network ID	Purpose
Ropsten	3	Old PoW-based testnet (now deprecated)
Rinkeby	4	PoA testnet (deprecated)
Kovan	42	PoA testnet (deprecated)
Goerli	5	Shared PoS testnet (replaced by Sepolia)
Sepolia	11155111	Current recommended testnet for developers
Holesky	17000	New large-scale PoS testnet (for validator testing)

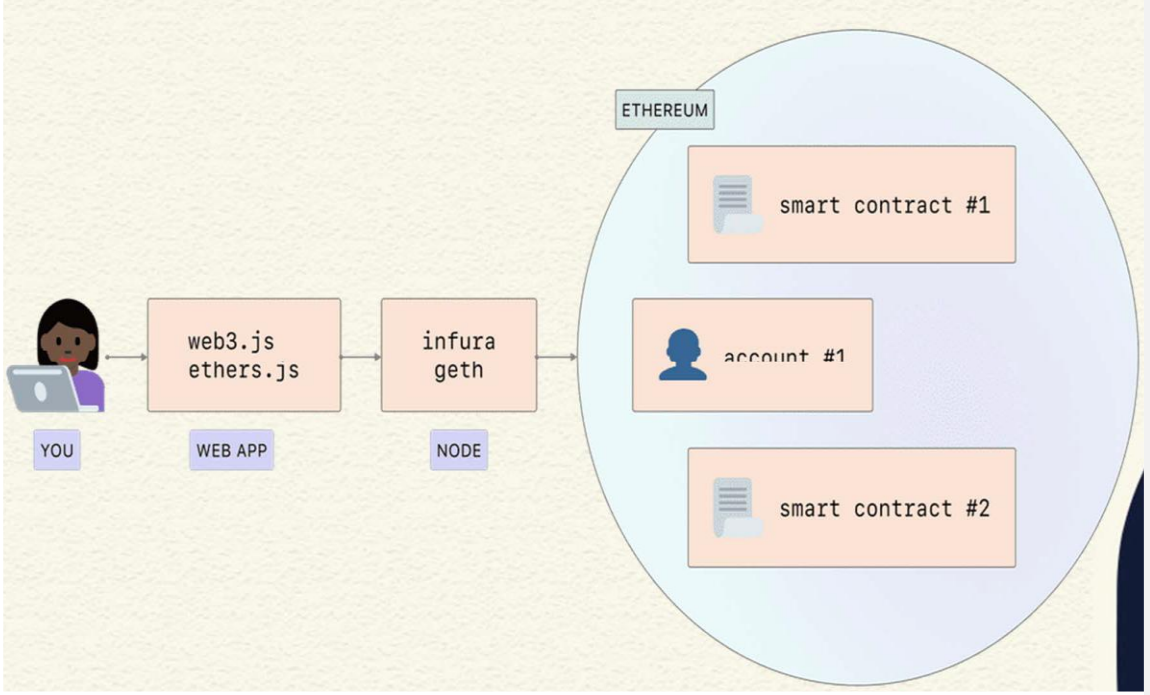
Components of the Ethereum Ecosystem

- The Ethereum stack showing various components
  1. Ethereum Network (Peer-to-Peer Layer)
  2. Ethereum Client (Node Software Layer)
  3. Local Blockchain Copy (Storage Layer)
  4. Web3.js or Custom Interface (Application Layer)



Ethereum ecosystem

—the components and flow that allow a user to interact with the Ethereum blockchain through decentralized applications (DApps)



- The **end user** who interacts with a **DApp (Decentralized Application)**
- **Web App Layer** is the **frontend application** the user interact with that connects to Ethereum using JavaScript libraries like: **web3.js** or **ethers.js**
- **Node Layer** represents the **Ethereum node** — the system that maintains a copy of the blockchain and broadcasts the transactions to the network. Nodes/node providers:- **geth, infura**
- **Ethereum Network** is the **core blockchain layer**, containing the **state of all accounts** and **smart contracts**

Elements of the Ethereum Blockchain

- Ethereum blockchain architecture consists of several high-level components, each contributing to Ethereum's operation
  - Keys and addresses
  - Accounts
  - Transactions and messages
  - Ether cryptocurrency/tokens
  - The EVM
  - Smart contracts

Ethereum – Keys and Addresses

- Ethereum uses **asymmetric cryptography** for ownership and identity.
- **Key Generation Process:**
  1. First, **a private key** is randomly chosen (256 bits positive integer) under the rules defined by elliptic curve secp256k1 specification
  2. The **public key** is then derived from this private key using **ECDSA** (Elliptic Curve Digital Signature Algorithm).
  3. An **address** is derived from the public key which is the right most 160 bits of the **Keccak-256 hash** of public key.

Ethereum Accounts

- Ethereum, the state is created or updated as a result of the interaction between accounts and transaction execution.
- An account will initiate a transaction to update the state of the Ethereum network.

- There are **two types** of accounts:
  - —**Externally Owned Accounts** (**EOAs**)
  - —**Contract Accounts** (**CAs**)

Types of accounts

- EOAs are similar to accounts that are controlled by a private key in Bitcoin.
- CAs are the accounts that have code associated with them along with the private key.
- CAs represent **smart contracts** stored and executed on the Ethereum blockchain.

Properties of each type of accounts

EOAs:

—EOAs has ether balance

—They are capable of sending transactions

—They have no associated code

—They are controlled by private keys

—Accounts contain a key-value store

—They are associated with a human user

CAs:

—Have an Ether balance.

—Contain code stored on the blockchain.

—Execute their code automatically when triggered by a transaction or message.

—Controlled by smart contract logic, not human users.

—Maintain their own storage (state) using a key-value store.

—Can call other contracts or trigger transactions programmatically.

—Execution is performed by the **Ethereum Virtual Machine (EVM)** on every node in the network.

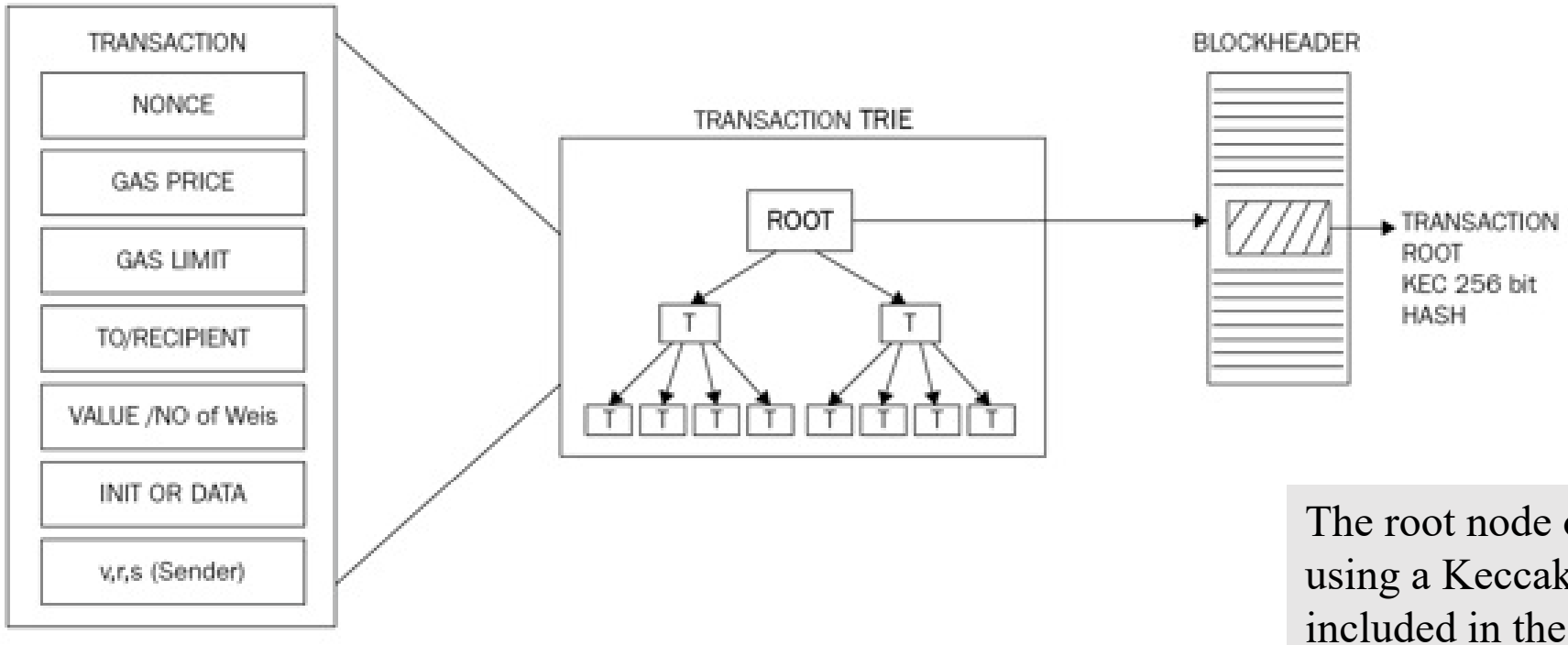
Transactions in Ethereum

- A transaction is a **digitally signed data packet** created and authorized using a **private key from accounts**
- It contains instructions that can either:
  - —Transfer Ether between accounts / calls a smart contract function, or
  - —Smart contracts creation
- Transactions can be divided into two types based on the output they produce:
  - **Message call transactions**:
    - The transactions simply produces a message call that is used to pass messages from one contract account to another.
  - **Contract creation transactions**:
    - The transactions result in the creation of a new contract account.
    - When this transaction is executed successfully, it creates an account with the associated code.

Structure of an Ethereum Transaction

Field	Description
Nonce	A counter that increments with every transaction sent by an account which ensures each transaction is unique.
Gas Price	The amount of Wei required to execute the transaction. This is the amount of Wei the user willing to pay for this transaction.
Gas Limit	The maximum amount of gas that can be consumed to execute the transaction. This is the amount of fee in ether that a user is willing to pay for computation.
To	The address of the recipient of the transaction. This is a 20-byte value.
Value	Value represents the total number of Wei to be transferred to the recipient; in the case of a contract account, this represents the balance that the contract will hold.
Signature	A digital signature generated using the sender's private key. It ensures transaction authenticity and allows recovery of the sender's public key.
Init / Data	For contract creation, `init` holds initialization code-used in the account initialization process. For message calls, `data` contains the input data of the message call.

Structure of an Ethereum Transaction



The root node of transaction trie is hashed using a Keccak 256-bit algorithm and is included in the block header along with a list of transactions in the block.

The relationship between transaction, transaction trie (a modified Merkle-Patricia tree) and block header

Contract Creation Transaction

- **Contract Creation Transaction** results in the **deployment of a new smart contract**

Parameter	Description
Sender	The account initiating the transaction
Original Transactor	The origin of the transaction; may differ in nested calls.
Available Gas	The amount of gas allocated to execute the transaction.
Gas Price	The price (in Wei per gas unit) the sender is willing to pay.
Endowment	The initial Ether balance sent to the new contract.
Byte Array (Init Code)	Contains the EVM code that runs during contract creation.
Initialization EVM Code	The code that initializes the contract — executed only once.
Current Stack Depth	Represents the nesting level of calls and creations.

Contract Address Generation

- The new **contract address** is **deterministically derived** using:

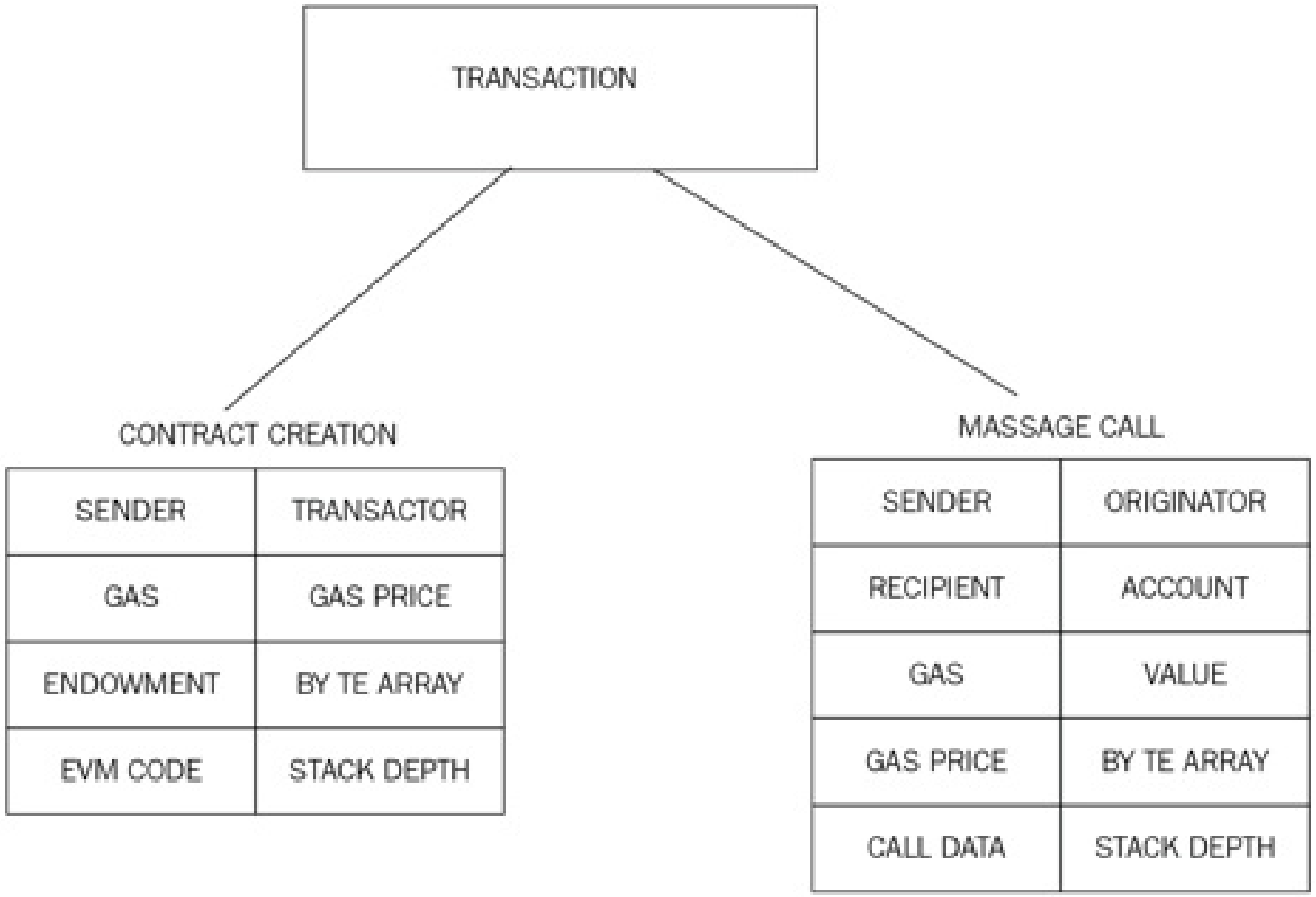
  Contract *Address*
  $= \textit{Rightmost 160 bits of } Keccak256(RLP(sender, nonce))$

- **RLP (Recursive Length Prefix)** is the encoding format used.
- **Sender** – the address creating the contract.
- **Nonce** – the sender's transaction count (incremented for each transaction).

Message Call Transaction

- A **Message Call Transaction** is a transaction that **calls or interacts with an existing contract** (or account)

Parameter	Description
Sender	Initiating account or contract.
Transaction Originator	The original EOA that started the chain of calls.
Recipient	The target account or contract being called.
Account Code Executed	The code belonging to the recipient, executed if present.
Available Gas	The gas allocated to this message execution.
Value	Amount of Ether transferred.
Gas Price	Cost per gas unit for execution.
Input Data	Data payload used for function parameters.
Stack Depth	Current execution depth of nested calls.


TRANSACTION
CONTRACT CREATION
SENDER
TRANSACTOR
GAS
GAS PRICE
ENDOWMENT
BY TE ARRAY
EVM CODE
STACK DEPTH
MASSAGE CALL
SENDER
ORIGINATOR
RECIPIENT
ACCOUNT
GAS
VALUE
GAS PRICE
BY TE ARRAY
CALL DATA
STACK DEPTH

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Wallet {
    uint public balance;

    function deposit() public payable {
        balance += msg.value; // increases balance
    }

    function getBalance() public view returns (uint) {
        return balance; // just reads balance
    }
}
```

Message Transaction (deposit)

When calls `deposit()` and sends 1 ETH:

- The transaction is **signed** by sender's private key.
- Sent to the **Ethereum network**.
- **Gas is consumed** to execute it.
- `balance` variable is **updated** in the blockchain state.
- It becomes part of the **mined block**.

Call Transaction(getBalance)

A user calls `getBalance()`:

- Executes **locally** on the node
- **No gas** is spent (because no miners or consensus involved).
- It **does not** broadcast to the network.
- Merely **reads** the state.

Ether Cryptocurrency / Tokens (ETH and ETC)

- Ethereum **native cryptocurrency called Ether (ETH)**.
- Ether plays a dual role:
  - It acts as a **medium of exchange** within the Ethereum ecosystem.
  - It serves as **"crypto fuel" (gas)** to power computations and smart contract executions on the Ethereum Virtual Machine (EVM).
- Ether (ETH) was the original token on the Ethereum blockchain.
- In 2016, the DAO hack led to a major split in the Ethereum community

Version	Description	Currency Symbol
Ethereum (ETH)	The hard-forked chain. Supported by most developers and continues active development.	**ETH**
Ethereum Classic (ETC)	The unforked, original chain that preserved the pre-hack history. Maintained by a smaller but dedicated community.	**ETC**

Ether in the Ethereum Ecosystem

- Ether is essential to the functioning of Ethereum.
- It serves multiple purposes:
  - **Reward for Miners/Validators**:
    Ether is minted as a reward for securing the network, validating transactions, and producing new blocks.
  - **Transaction Fees (Gas)**:
    Every operation in Ethereum requires computational power, measured in **gas**. Users pay gas fees in Ether to incentivize miners/validators.
  - **Computation and Smart Contracts**:
    Ether is consumed as “fuel” to execute operations in the **EVM (Ethereum Virtual Machine)** — from simple transactions to complex contract executions.

Ether Denominations

- The smallest unit is **Wei**, named after cryptographer Wei Dai.

Unit	**Alternative Name**	**Exponential Value (Wei)**	**Number of Wei**
Wei	Wei	10^0	1
KWei	Babbage	10^3	1,000
MWei	Lovelace	10^6	1,000,000
GWei	Shannon	10^9	1,000,000,000
Micro Ether	Szabo	10^{12}	1,000,000,000,000
Milli Ether	Finney	10^{15}	1,000,000,000,000,000
Ether	Ether	10^{18}	1,000,000,000,000,000,000

The Ethereum Virtual Machine (EVM)

- The core computational engine
- It is responsible for executing smart contracts and managing state transitions
- EVM → State Transition Machine
  - The EVM takes an **input state** and a **set of instructions (bytecode)**.
  - It executes the bytecode **instruction by instruction**, producing a **new state**.
  - This transformation is deterministic — meaning every node that executes it will reach the same final result.

Components of EVM Architecture

Stack

- Operates on a **LIFO** principle.
- Used to hold temporary values for computations.
- Each item is 256 bits wide.

Memory

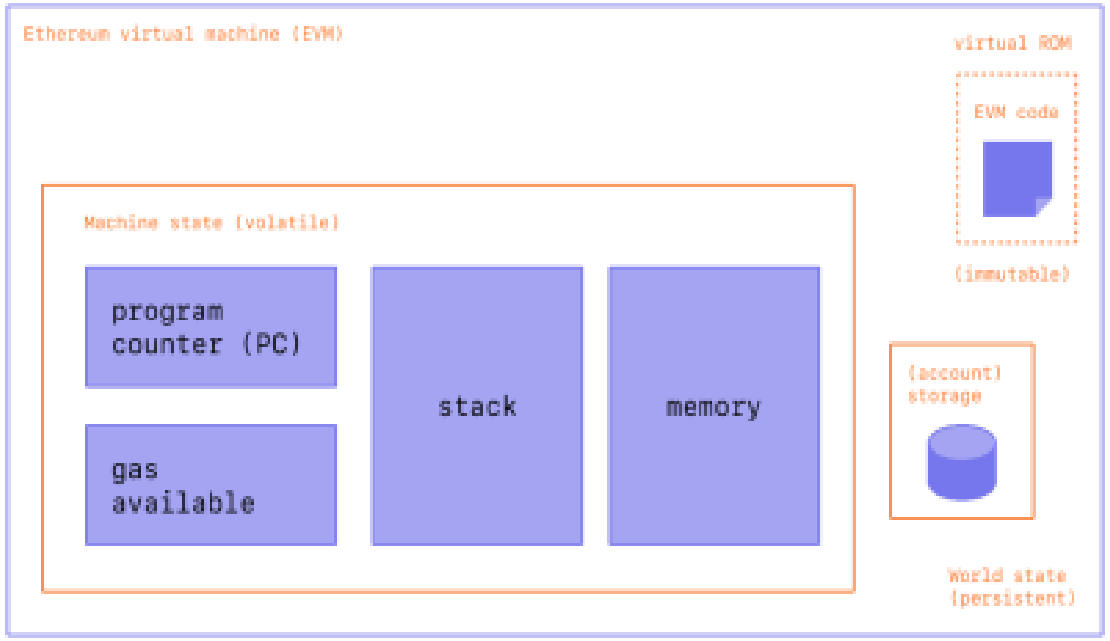
- Temporary storage (like **RAM**).
- Exists only during the execution of a contract.
- Cleared once the function or transaction finishes.
- Represented as a **byte array**.

Storage

- **Permanent key-value store** associated with each contract.
- Remains persistent between executions.
- Each key and value is **32 bytes**.
- Comparable to **hard disk storage**.

Virtual ROM

- Contains the **bytecode** of the contract.
- Accessed via the CODECOPY instruction, which copies the program code into memory before execution.



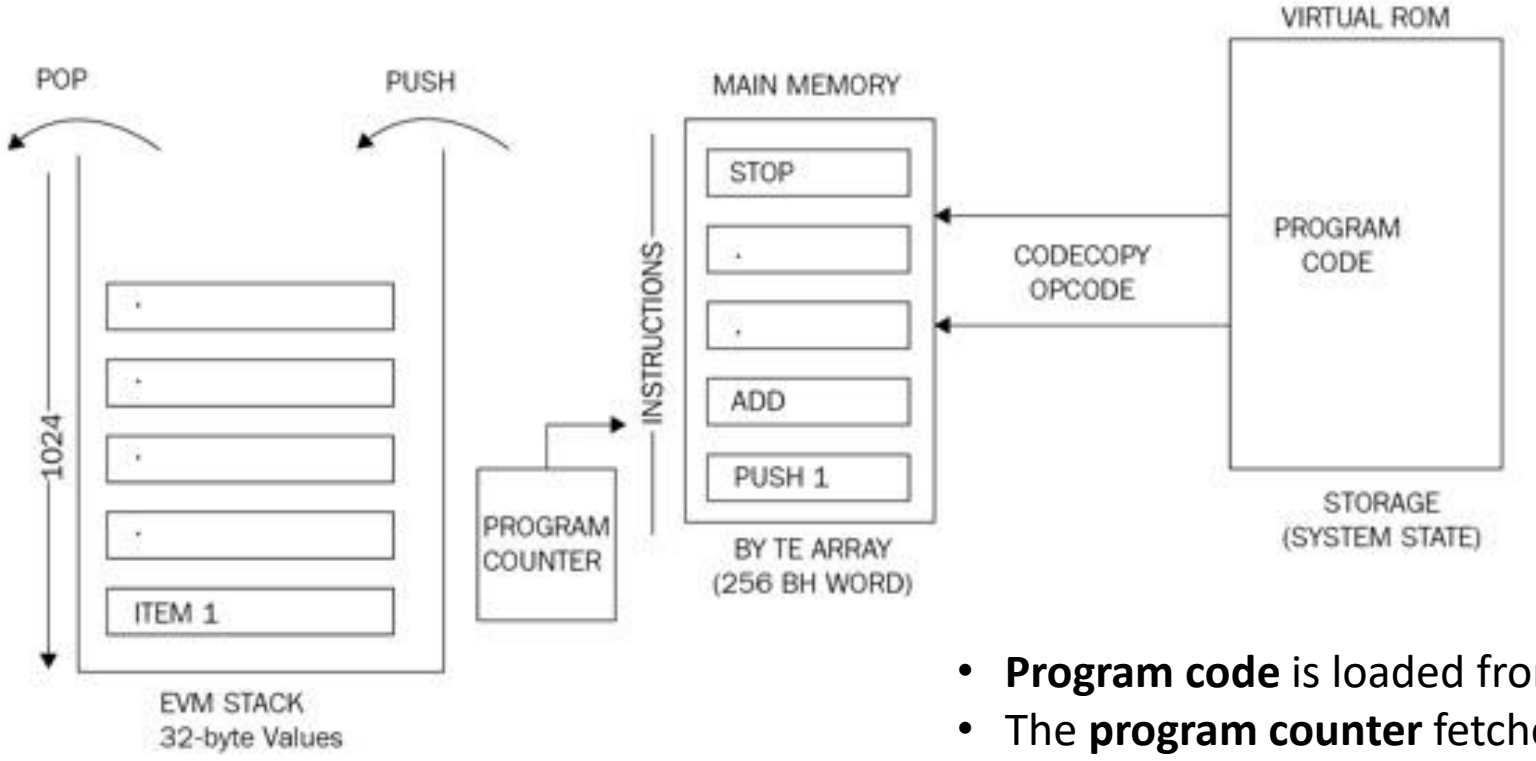
Program Counter (PC)

- Keeps track of the **current instruction** being executed.
- Automatically increments with each operation.

Gas

- Each EVM operation consumes a certain amount of gas.
- Prevents **infinite loops** and **Denial of Service (DoS)** attacks.
- When gas runs out, execution halts, and all state changes are reverted (except gas fee deduction).

EVM Operation



- **Program code** is loaded from virtual ROM into main memory.
- The **program counter** fetches instructions sequentially.
- Instructions manipulate the **stack**, **memory**, or **storage**.
- Gas is deducted for each operation.
- Execution halts when:
  - STOP, RETURN, or SELFDESTRUCT opcode is encountered.
  - An exception occurs (e.g., invalid instruction or out-of-gas).

Smart Contract Lifecycle in Ethereum

- A **smart contract** is a **self-executing program** stored on the Ethereum blockchain that runs exactly as programmed without the need for intermediaries.
- Smart contracts are written in **high-level programming languages** like **Solidity** and executed by the **Ethereum Virtual Machine (EVM)**.

Smart Contract Code (Solidity)

- Solidity is a **high-level, contract-oriented** programming language designed for writing smart contracts on Ethereum.
- A Solidity program defines:
  - Functions (logic and operations).
  - State variables (data storage).
  - Events (for external applications).
  - Access modifiers (like public, private, etc.).

Compilation Process

- When Solidity code is compiled (using the **Solidity compiler** solc or an IDE like **Remix**), two outputs are generated:

(a) Bytecode

- This is **low-level machine code** (in hexadecimal form) that the **EVM** can execute.
- The bytecode represents the actual logic that will be deployed to and executed by the Ethereum blockchain.
- It contains **EVM opcodes**
- Each opcode corresponds to a specific computation or storage operation inside the EVM.

(b) Application Binary Interface (ABI)

- The **ABI** defines **how to interact** with the compiled contract.
- It is a **JSON representation** of:
  - The contract's functions.
  - Input and output parameters.
  - Events and types.

Deployment on the Ethereum Network

- The **bytecode** is sent to the Ethereum blockchain through a **transaction**.
- When the transaction is confirmed (mined), the contract is stored permanently on the blockchain.
- The contract is assigned a **unique address** (called CA).
- Once deployed, the contract becomes **immutable** — meaning its logic cannot be changed.

Interaction with Smart Contracts

- After deployment, users and applications interact with the contract through its **ABI** using tools like **Web3.js**
- Web3.js is a **JavaScript library** that connects web applications to the Ethereum blockchain.
- Allows developers to:
  - Call contract functions.
  - Send transactions.
  - Read blockchain data.

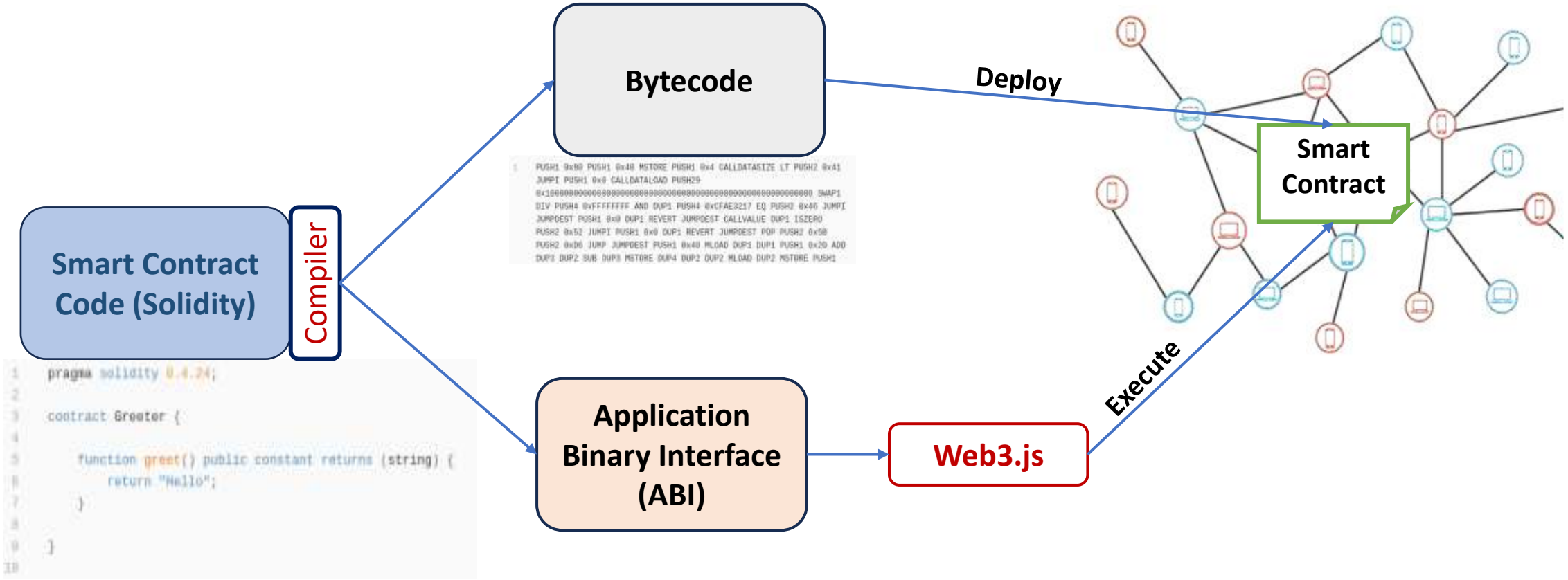
Smart Contract (Solidity)

```
pragma solidity ^0.8.7;
contract Greeter {
    function greet() public pure returns (string memory) {
    return "Hello";
  }
}
```

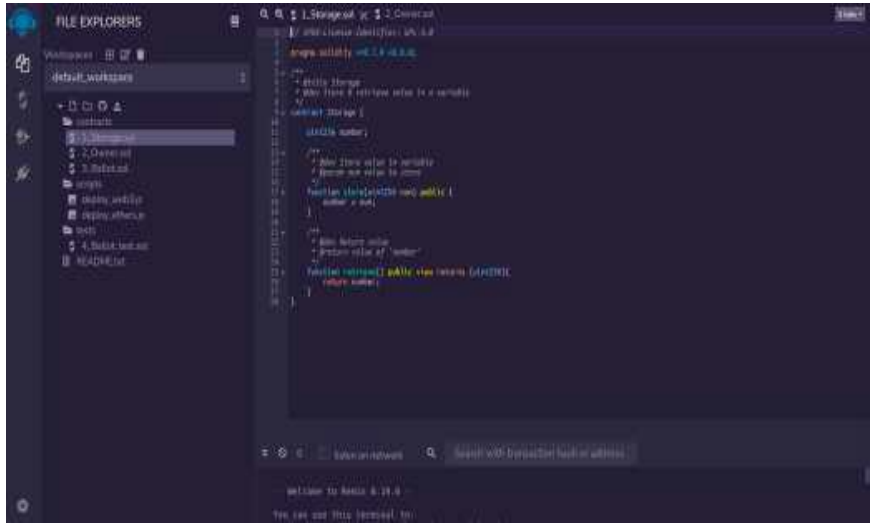
JavaScript

```
const contract = new web3.eth.Contract(abi, contractAddress);
const message = await contract.methods.greet().call();
console.log(message); // Output: "Hello"
```

Compiler


Smart Contract Code (Solidity)
Compiler
Bytecode
Application Binary Interface (ABI)
Web3.js
Deploy
Execute
Smart Contract
pragma solidity 0.4.24;
contract Greeter {
function greet() public constant returns (string) {
return "Hello";
}
}

Compiling Solidity



solc compiler

- Compile .sol files to bytecode
- Command line tool

https://docs.soliditylang.org/en/v0.8.9/installing-solidity.html#installing-solidity

Remix editor

- Browser based
- Compile, emulate, deploy contracts
- https://remix.ethereum.org/

Module IV

Hyperledger – Reference architecture-Requirements and design goals of hyper ledger fabric – Distributed ledger – proof of Elapsed Time – Transaction families – Attacks on blockchain – Applications on block challenges

Hyperledger Foundation

- Open source community -focused on **enterprise-grade blockchain deployments**.
  - Different companies / organizations want to collaborate
  - Closed group: members know each other
  - Do not fully trust each other
  - Distributed shared ledger –based on permissioned consensus
- Home for various distributed ledger frameworks including: Hyperledger Fabric, Sawtooth, Indy, etc.

https://www.hyperledger.org/

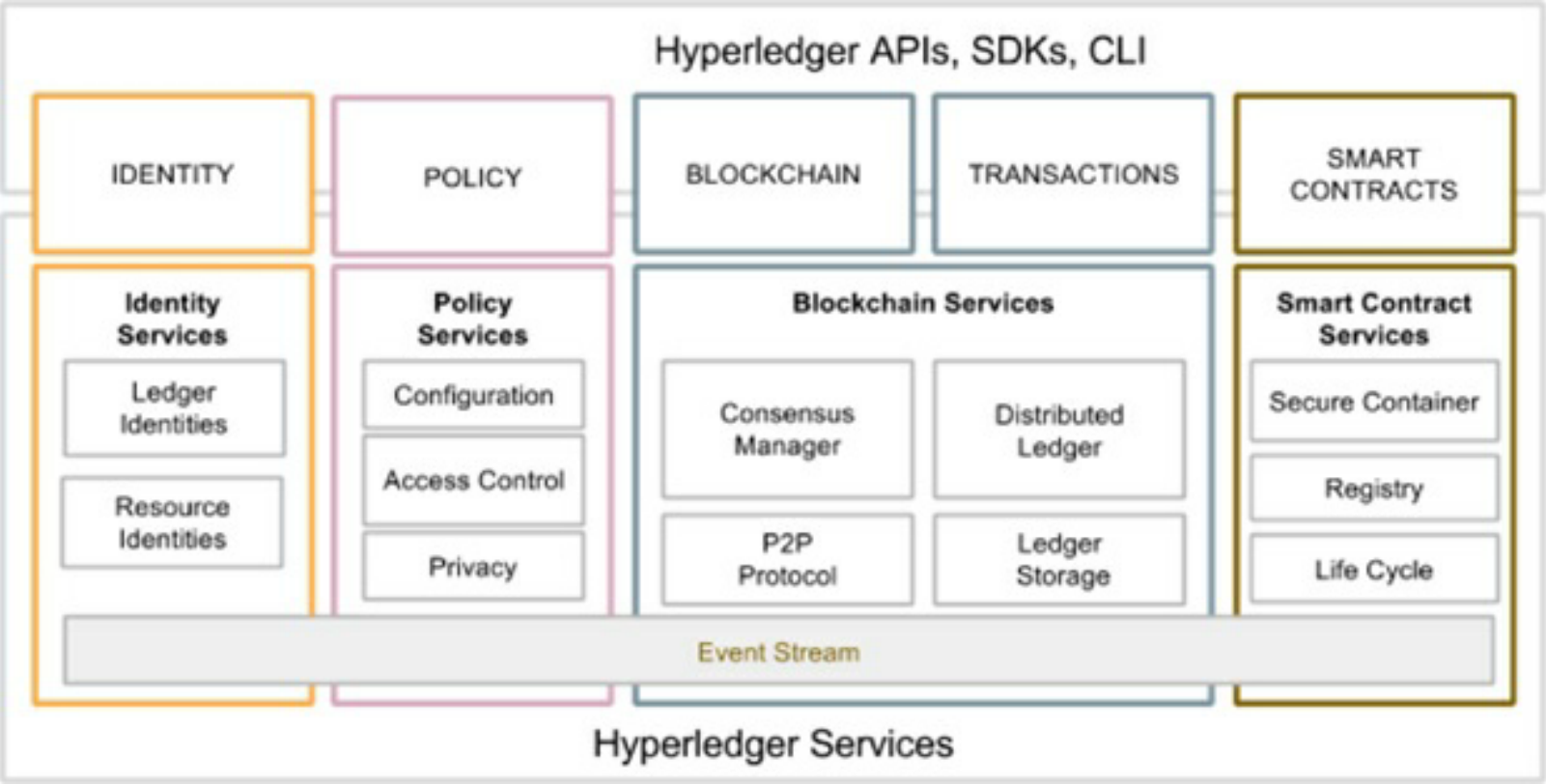
The Linux Foundation Hyperledger Project

HYPERLEDGER FABRIC
HYPERLEDGER GRID
HYPERLEDGER INDY
HYPERLEDGER IROHA
HYPERLEDGER QUILT
HYPERLEDGER SAWTOOTH
HYPERLEDGER TRANSACT
HYPERLEDGER URSA
HYPERLEDGER ARIES
HYPERLEDGER AVALON
HYPERLEDGER BESU
HYPERLEDGER BURROW
HYPERLEDGER CACTUS
HYPERLEDGER CALIPER
HYPERLEDGER CELLO
HYPERLEDGER EXPLORER

The reference architecture

Purpose of the reference architecture:

- Provide a **common framework** for blockchain platforms
- Define core blockchain **components**
- Support **modular and pluggable** design
- Enable **enterprise blockchain** development



The reference architecture

- Provides a **standard blueprint or guideline** for designing and implementing blockchain systems.
- The Hyperledger project, developed by the Linux Foundation, published a white paper describing a reference architecture for permissioned blockchains.
- This architecture helps developers build **enterprise-grade distributed ledger systems**.
- The document describing this architecture is available in the **Hyperledger white paper**.

The reference architecture

- The Hyperledger reference architecture consists of **five major top-level components**.
- Main components include:
  - Identity
  - Policy
  - Ledger
  - Transactions
  - Smart Contracts

Identity Component

- The **Identity component** manages **membership services** within the blockchain network.
- In a **permissioned blockchain**, only **authorized participants** can join and perform operations.
- Functions of Identity Services
  - Identification of participants
  - Authentication of users
  - Authorization of operations
  - Membership management
- Identity services ensure that:
  - Only **trusted organizations** participate in the network
  - Participants have **verified digital identities**
- Example:
  - A supply chain blockchain may allow participation from:
    - Manufacturer
    - Distributor
    - Retailer
    - Logistics provider
- Each participant must first obtain **verified credentials** from the identity service.

Policy Component

- The **Policy component** defines rules governing the blockchain network.
- Policies determine:
  - Who can perform specific operations
  - How transactions are validated
  - What conditions must be satisfied before adding a block
- **Examples of Policies**

  1. Endorsement policy
  - Specifies which nodes must approve a transaction before it becomes valid.
  - Example:
    - A transaction may require approval from:
    - Manufacturer node
    - Logistics node

  2. Consensus policy
  - Defines how participants agree on the **order of transactions**.
  - Policies help maintain:
    - Governance
    - Trust
    - Compliance

Ledger and Transactions Component

- The ledger and transactions component forms the core of the blockchain system.
- It contains:
  - Distributed ledger:- stores all blockchain data.
  - Ordering services:- The ordering service determines the sequence of transactions in the blockchain.
  - Network protocols
  - Endorsement and validation mechanisms:- Selected nodes execute the transaction proposal and sign the result as proof. The system checks whether the transaction satisfies the policy rules

Smart Contracts Layer

- The **Smart Contract layer** implements **business logic**.
- In Hyperledger, smart contracts are called **Chaincode**.
- **Responsibilities**
  - Define business rules
  - Execute transactions
  - Modify ledger state
- **Secure Container Technology:-** Smart contracts are executed inside **secure containers**.

Requirements and Design Goals of Hyperledger

- The key design goals are:
  - Modularity
  - Privacy and confidentiality
  - Scalability
  - Deterministic transactions
  - Identity management
  - Auditability
  - Interoperability
  - Portability
  - Rich data queries

Modular Architecture

- One of the most important design goals of Hyperledger is **modularity**.
- Modularity means that different components of the blockchain system are designed as **independent modules** that can be added, removed, or replaced without affecting the entire system.
- The following components can be modular and pluggable:
  - Storage system
  - Consensus mechanism
  - Policy management
  - Chaincode execution
  - Access control
  - Cryptographic services

Privacy and Confidentiality

- Privacy and confidentiality are **critical requirements in enterprise blockchain systems**.
- Unlike public blockchains, enterprise systems often require that **only authorized participants can view specific transactions**.
- Key Features Supporting Privacy
  - Access control mechanisms
  - Advanced cryptographic algorithms
  - Secure identity management
  - Permissioned network participation

Scalability

- Scalability refers to the ability of the system to **handle a large number of transactions and users efficiently**.
- Enterprise blockchain applications may involve:
  - Thousands of transactions per second
  - Multiple organizations
  - Large-scale networks
- Hyperledger aims to provide:
  - High transaction throughput
  - Support for large network sizes
  - Efficient resource utilization

Deterministic Transactions

- A deterministic transaction means that **the same transaction always produces the same result when executed by different nodes**.
- Deterministic execution is essential for:
  - Achieving consensus
  - Maintaining ledger consistency
  - Preventing conflicting results

Identity Management

- Enterprise blockchains require **strong identity management systems**.
- Unlike public blockchains where users are anonymous, Hyperledger operates in a **permissioned environment**.
- The system must support:
  - User identification
  - Authentication
  - Authorization
  - Access control
- Hyperledger uses **PKI-based identity systems** where participants receive **digital certificates**.
- These certificates allow nodes to:
  - Verify identities
  - Authenticate transactions
  - Control network participation

Auditability

- Auditability refers to the ability to **track and verify all activities within the blockchain network**.
- Features Supporting Auditability
  - Immutable transaction records
  - Complete history of operations
  - Traceability of identities
- Audit trails help organizations:
  - Detect fraud
  - Perform regulatory audits
  - Monitor system activity

Interoperability

- Currently, many blockchain platforms exist, but most operate **independently and cannot communicate with each other**.
- This limitation restricts the development of a **global blockchain ecosystem**.
- Interoperability Goal
  - Hyperledger aims to support **communication between different blockchain networks**.
- This requires:
  - Standardized protocols
  - Cross-platform communication mechanisms
  - Data exchange frameworks

Portability

- Portability refers to the ability of the blockchain system to **run across different platforms and environments without code modification**.
- Hyperledger is designed to support portability at multiple levels:
  - Infrastructure level
  - Code level
  - Library level
  - API level
- Portability allows developers to:
  - Deploy applications across different platforms
  - Reuse code in multiple environments
  - Reduce development complexity

Rich Data Queries

- Enterprise applications require the ability to **perform complex data queries on the blockchain**.
- Hyperledger Fabric supports **rich query capabilities** that allow users to retrieve data efficiently.
- Users can query:
  - Current state of assets
  - Transaction history
  - Ledger data

Distributed Ledger

- A **distributed ledger** is a shared database that is **replicated across multiple nodes in a blockchain network**.
- Every participant in the network maintains a copy of the ledger, ensuring transparency, integrity, and fault tolerance.
- In Hyperledger Fabric, the distributed ledger consists of **two main components**:
  - **Blockchain**
  - **World State**
- These two elements together maintain the **history of transactions** and the **current state of assets** in the network.

Components of the Distributed Ledger

Blockchain

- The **blockchain** is a **cryptographically linked list of blocks**.
- Each block contains a set of transactions and is connected to the previous block through a hash.
- Characteristics of the blockchain:
  - Immutable (cannot be altered once written)
  - Sequentially ordered blocks
  - Cryptographically linked
  - Provides complete transaction history
- The blockchain records **all transactions that have occurred in the network**.
- Each new block added to the blockchain contains the **hash of the previous block**, which ensures the **integrity and security of the ledger**.

World State

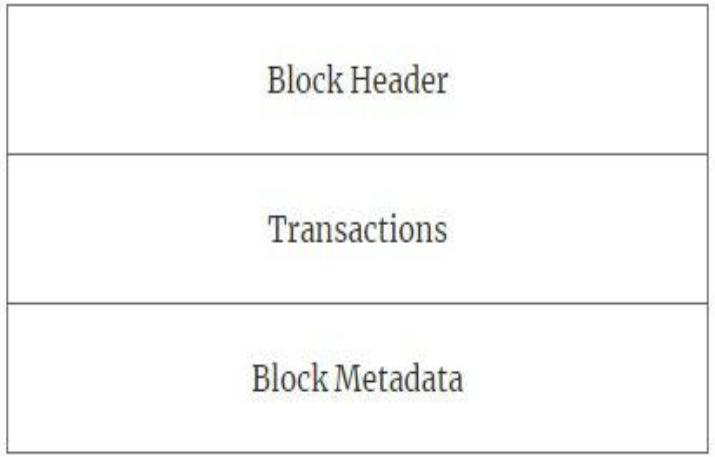
- The **world state** represents the **current state of the ledger**.
- It is implemented as a **key-value database** where:
  - **Key** represents an asset identifier
  - **Value** represents the current state of that asset
- Smart contracts interact with the world state during transaction execution.
- Functions of world state:
  - Store the latest value of assets
  - Allow efficient querying
  - Enable quick updates by transactions
- Unlike blockchain, which stores the **full history**, the world state only stores the **latest state of data**.

World State Storage

- Each node in the network stores the world state locally using databases such as:
  - **LevelDB**
  - **CouchDB**
- **LevelDB**
  - Default key-value database
  - Simple and efficient
  - Suitable for basic queries
- **CouchDB**
  - Document-based database
  - Supports JSON data
  - Enables rich queries
- Because Hyperledger supports **pluggable data stores**, organizations can select the database that best fits their application

Blocks in Hyperledger

- The blockchain consists of **blocks**, and each block contains a **set of transactions**.
- A block in Hyperledger consists of **three main components**:
  - Block Header
  - Transactions (Data)
  - Block Metadata



Block Structure

Block Header

- The **block header** contains metadata required to identify and link blocks in the blockchain.
- It consists of three important fields:

  Block Number - represents the **position of the block in the blockchain sequence**.

  Previous Hash - field contains the **hash of the previous block header**.

  Data Hash - The **data hash** represents the cryptographic hash of the **transactions stored in the block**.

Block Structure

Transactions (Block Data)

- The **transaction section** of the block contains the actual operations executed on the blockchain.
- Transactions typically include **chaincode execution requests**.
- Transactions are generated when a client application interacts with **smart contracts (chaincode)**.
- Basic Transaction Fields

  Transaction Type – type of transaction being executed
  Version – protocol version used
  Timestamp – time at which transaction was created
  Channel ID – blockchain channel where transaction is executed
  Transaction ID – unique identifier for each transaction
  Epoch – configuration version indicator

Block Structure

- Other Transaction Fields
  - Chaincode Path – location of the smart contract
  - Chaincode Name – name of the deployed chaincode
  - Chaincode Version – version of the smart contract
  - Chaincode Type – programming language or execution type
  - Creator Identity – identity of the user submitting the transaction
  - Signature – cryptographic signature verifying authenticity
  - Input – parameters passed to the smart contract
  - Timeout – execution timeout for the transaction
  - Endorser identities and signatures
  - Proposal hash
  - Chaincode Events – events generated during execution
  - Response Status – success or failure of transaction
  - Namespace – logical grouping of state variables
  - Read Set – list of keys read during transaction execution
  - Write Set – list of keys updated during execution

Block Structure

Block Metadata

- The **block metadata** contains information related to **validation, signatures, and configuration**.

  Creator Identity:- Identifies the node or service responsible for creating the block.

  Signatures:- Digital signatures from network participants confirming: Block authenticity and Transaction approval

  Last Configuration Block Number:- Stores the block number where the latest network configuration change occurred.

  Transaction Validation Flags:- flag(valid/invalid) for each transaction included in the block

  Last offset persisted:- Kafka-based ordering services, metadata stores the last persisted offset.

Proof of Elapsed Time (PoET) in Hyperledger

- **Proof of Elapsed Time (PoET)** is a consensus algorithm designed for permissioned blockchain networks.
- It randomly selects a node to create the next block based on a **random waiting time**.
- PoET does **not require heavy computational power or energy consumption**.
- Instead, it uses **trusted hardware technology** to ensure fairness and randomness in selecting the block creator.

Key Idea of PoET

- The main idea behind PoET is **leader election through random wait times**.
- **Process Overview**
  - Every participating node requests a **random wait time** from a trusted execution environment.
  - Each node begins waiting for its assigned time.
  - The node whose timer expires **first wins the right to create the next block**.
  - That node broadcasts the block to the network.
  - Other nodes verify the waiting certificate and accept the block.

Trusted Execution Environment (TEE)

- PoET uses **Intel Software Guard Extensions (SGX)** to create a secure environment for generating random wait times.
- Role of SGX
  - SGX provides:
    - Secure code execution
    - Cryptographic verification
    - Protection from tampering
- The blockchain framework **Hyperledger Sawtooth** originally implemented PoET.

Transaction Families

- In many blockchain platforms, smart contracts run inside a **general-purpose virtual machine**.
- For example:- Ethereum Virtual Machine in Ethereum.
- This approach allows developers to build **any application**, but it also introduces security risks.

Problem with Traditional Smart Contract Model

- A **generic virtual machine model**:
  - Provides a powerful programming environment
  - Allows arbitrary operations
  - Increases the **attack surface**
  - This has led to several blockchain vulnerabilities.

Example

- A major attack occurred in **The DAO Hack**, where attackers exploited smart contract vulnerabilities.

Solution: Transaction Families

- To reduce risks, Hyperledger introduced the concept of Transaction Families.
- A **transaction family** is a **domain-specific smart contract system**.
- It separates:
  - **Business logic**
  - **Data model**
  - **Transaction rules**
- This makes blockchain applications **more secure and specialized**.

Structure of Transaction Families

- A transaction family consists of two main components:

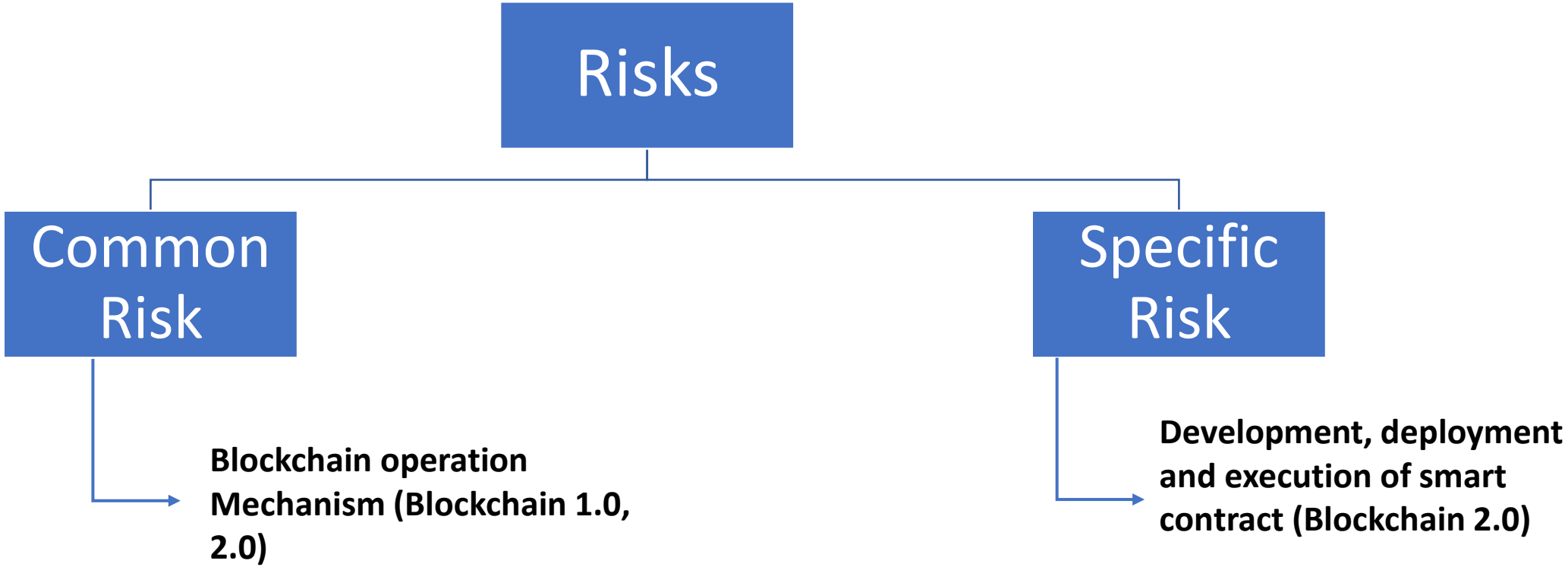
1. Data Model

- Represents the **current state of the blockchain ledger**.
- Example:
  - Account balances
  - Asset ownership
  - Supply chain records

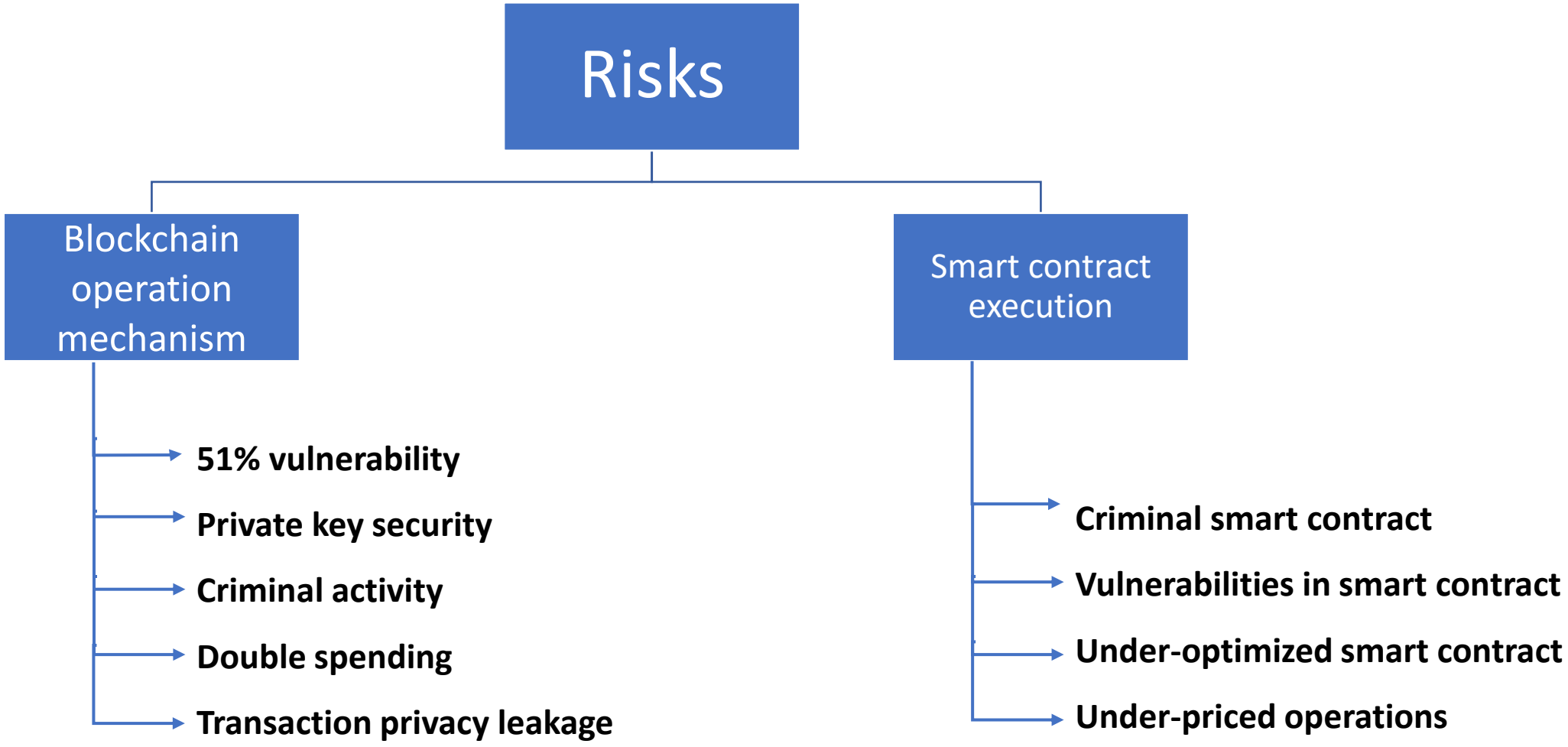
2. Transaction Language

- Defines how the state **can be modified**.
- Examples:
  - Transfer asset
  - Update record
  - Register participant
- **Domain-Specific Logic**
  - Transaction families are designed for **specific industries or domains**.

Risks in Blockchain



Risks in Blockchain



Risks in Blockchain

1. 51% Vulnerability

- Occurs when a single entity controls **more than 50% of the network's mining/validation power**
- Attacker can:
  - Reverse transactions
  - Prevent new transactions from confirmation
- Loss of trust and network integrity

2. Private Key Security

- Blockchain ownership depends on **private keys**
- If a key is:
  - Lost → assets are permanently inaccessible
  - Stolen → attacker gains full control
- Key Issue is No central authority to recover keys

Risks in Blockchain

3. Criminal Activity

- Pseudonymous nature enables:
  - Money laundering
  - Illegal trade
  - Ransomware payments
- **Challenge:** Difficult to trace real identities

4. Double Spending

- Same digital asset is spent more than once
- Typically prevented by consensus, but possible in:
  - Weak networks
  - 51% attacks
- **Impact:** Financial fraud

5. Transaction Privacy Leakage

Transactions are publicly visible on blockchain

Even if identities are hidden:

Patterns can reveal user behavior

Risk: Loss of financial privacy

Risks in Blockchain

1. Criminal Smart Contracts

- Smart contracts can be programmed for:
  - Illegal marketplaces
  - Automated scams
- **Issue:** Once deployed, difficult to stop

2. Vulnerabilities in Smart Contracts

- Bugs in code can be exploited
- **Examples:**
  - Reentrancy attacks
  - Overflow/underflow errors
- **Impact:** Loss of funds (e.g., major hacks in blockchain history)

3. Under-Optimized Smart Contracts

- Inefficient code leads to:
  - High execution cost (gas fees)
  - Slow performance
- **Impact:** Increased operational expenses

4. Under-Priced Operations

- Incorrect gas pricing or design flaws
- Attackers exploit low-cost operations to:
  - Flood the network
  - Cause denial-of-service (DoS)

Simple Use Cases by Industry

Financial Services	**Public Sector**	**Retail**	**Insurance**	**Supply Chain & Logistics**
• Trade Finance • Cross currency payments • Mortgages • KYC • Cross border tax	• Asset Registration • Citizen Identity • Medical records • Medicine supply chain	• Supply chain • Loyalty programs • Information sharing (supplier – retailer)	• Claims processing • Risk provenance • Asset usage history • Claims file	• Supply chain finance • Maintenance tracking • Provenance • Supply chain compliance

What makes a good blockchain use case?

- Identifying a good blockchain use-case is not always easy!
  - However, there should always be:

    1. A **business problem** to be solved
       - That cannot be more efficiently solved with other technologies
    2. An identifiable **business network**
       - With Participants, Assets and Transactions
    3. A need for **trust**
       - Consensus, Immutability, Finality or Provenance

Understanding the Business Problem

1. What is the specific business problem / challenge that the project will address?
   - Scope the business challenge up front
2. What is the current way of solving this business problem?
   - Understand current systems and areas for improvement

3. Assuming the business problem is large, what specific aspects of this business problem will be addressed?

Understanding the Participants

1. Who are the business network participants (organizations) involved and what are their roles?
    - If there is no business network involved, then this is not a good use case
2. Who are the specific people within the organization and what are their job roles?
    - Understand the key users in a business network.

- Who are the participants? How many types of participants?
- How will they access and interact with the blockchain?
- Will they be peer nodes?
- Do you need web or mobile apps?
- Are gateways (such as exchanges or data providers) needed?
- Do you need to integrate to external data sources?
- Who will operate the blockchain? Who will govern/regulate the blockchain?
- What is the value/incentive for each participant to join the network?

Identities

- Do you need to know your users?
  - Pseudo-anonymous blockchain like bitcoin does not require user identities to beverified
- In most business use-cases, some form of identity is required
  - In public blockchains, an identity oracle (linked to a trusted database) could providesuch information sources
    - Sources can come from governments, financial institutions or utility providers
  - In private blockchains, a gateway or controller ensures identity is verified beforecredentials are issued to the user
    - Decentralized identity management is also possible –we have seen that –may be the preferredway for a blockchain application

Understanding the Assets and Transactions

1. What assets are involved and what is the key information associated with the assets?
2. What are the transactions involved, between whom, and what assets are associated with transactions?
   - Understand under what business or contractual conditions assets are under,as they transfer from one owner to another.

Defining Transactions

- What types of processes need to take place in your blockchain network?
  - Invoke actions –add,delete, change, transfer
  - Query
  - Do you need to control access to these functions based on participant types or roles?

Supply Chain Management (SCM)

The most prominent use case for enterprise blockchains like **Hyperledger Fabric**.

- **The Problem:** Traditional supply chains are opaque. If a food product is contaminated, it takes weeks to trace the source through paper trails and siloed databases.
- **The Solution:** Every step of the journey (Manufacturing →Shipping →Warehouse → Retail) is recorded as a transaction.
- **Mechanism:** Using the **Execute-Order-Validate** model, sensors (IoT) can automatically push data (like temperature or location) to the blockchain.
- **Benefits:** Real-time provenance, reduced fraud, and immediate identification of contaminated batches.

Healthcare Data Management

- **The Problem:** Patient records are scattered across different hospitals. Sharing data is slow and risks privacy breaches.
- **The Solution:** A decentralized ledger stores a "hash" (digital fingerprint) of the medical record. The actual file stays in secure storage, but the blockchain manages **access permissions**.
- **Role of Smart Contracts:** Contracts can automatically grant a doctor access to a patient’s history only during an emergency or after the patient provides a digital signature via their **EOA (Externally Owned Account)**.

Decentralized Finance (DeFi)

- This application leverages the **Ethereum Ecosystem** and **Smart Contracts**.
- **The Concept:** Replacing banks with code.
- **Applications:**
  - **Lending/Borrowing:** Users deposit crypto into a contract to earn interest or use it as collateral for a loan.
  - **Decentralized Exchanges (DEXs):** Trading tokens directly via automated liquidity pools (e.g., Uniswap) rather than a central order book.
- **Components involved:** Uses **EVM** for execution and **World State** to track global balances in real-time.

Digital Identity (Self-Sovereign Identity)

- **The Problem:** We rely on centralized providers (Google, Facebook, Governments) to prove who we are. If their servers are hacked, our identity is stolen.
- **The Solution:** A blockchain-based identity where the user holds their private keys.
- **Mechanism:** You provide a "Zero-Knowledge Proof" to a service provider (proving you are over 18 without revealing your actual birth date). The blockchain validates the claim without storing the sensitive data itself.

Solidity Programming Language

- **Solidity** is a **domain-specific programming language** used for writing **smart contracts on the Ethereum blockchain**.
- A smart contract written in Solidity is compiled into **bytecode**, which runs on the **Ethereum Virtual Machine (EVM)**.
- Solidity is the most widely used language for Ethereum smart contract development.
- Its syntax is similar to JavaScript, C, and C++.
- It is designed specifically for contract-oriented programming.
- Solidity is a **statically typed language**, meaning:
  - Every variable must have a **declared data type**
  - Type checking happens **at compile time**

```
uint x = 10;
bool valid = true;
```

Contract-Oriented Language

- Solidity is called **contract-oriented** because programs are structured around **contracts**.
- A **contract** in Solidity is similar to a **class in object-oriented programming**.
- A contract can contain:
    - State variables
    - Functions
    - Events
    - Structs
    - Modifiers

```
contract MyContract {
    uint value;
}
```

Solidity Data Types

- Solidity supports two main categories of data types:
  - **Value Types**
  - **Reference Types**
- Value types store **data directly** and are passed **by value**.

1. Boolean

- Boolean values represent **true or false**.
- Example:

```
bool v = true;
bool v = false;
```

Solidity Data Types

2. Integers

- Solidity supports **signed and unsigned integers**.

Type	Description
int	Signed integers
uint	Unsigned integers

```
int8 x;
int16 y;
int256 z;
uint8 a;
uint16 b;
uint256 c;
```

Solidity Data Types

3. Constant Variables

- Constants do not reserve storage in the blockchain.

```
uint constant z = 10 + 10;
```

4. Address Type

- The **address data type** stores an Ethereum address.
- Address size = **160 bits (20 bytes)**

```
address user;
```

Important Address Members

i. balance

- Returns the balance of the address in Wei.

```
user.balance
```

ii. send()

- Used to send Ether to an address.

```
address to;
to.send(20);
```

Solidity Data Types

5. Byte Arrays

- Solidity supports **fixed-size and dynamic byte arrays**.
- **Fixed-size bytes**
- Range from **bytes1 to bytes32**

```
bytes32 data;
```

- **Dynamic bytes**

```
bytes data;
string name;
```

- Eg:-

```
bytes32[10] bankAccounts;
bytes32[] trades;
```

Solidity

6. Literals

- Literals represent **fixed values in the code**.
- **Integer Literals**

```
uint8 x = 2;
```

- **String Literals**
- Strings can be written using **single or double quotes**.

```
"packt"
'packt'
```

- **Hexadecimal Literals**
- Hex values start with **hex keyword**.

```
hex"AABBCC"
```

Solidity

7. Enums

- Enums allow developers to create **custom data types**.

```
enum Order {Filled, Placed, Expired}
```

8. Function Types

- Solidity supports two main function types.
  - **Internal Functions**
    - Used **only within the contract**.
  - **External Functions**
    - Can be called **from outside the contract**.

```
function example(uint x) external returns(uint)
```

Solidity

Reference Types

- Reference types store **data by reference instead of value**.
- These include:
  - Arrays
  - Structs
  - Mappings
- Arrays store **multiple elements of the same type**. uint[] orderIds;
- Structs allow grouping **multiple variables of different types.**

```
struct Trade
{
    uint tradeid;
    uint quantity;
    uint price;
    string trader;
}
```

Solidity

Data Location

- Solidity specifies where data is stored.

Location	Description
storage	permanent blockchain storage
memory	temporary storage
calldata	read-only function arguments

```
function store(uint[] memory data)
```

Mappings

- Mappings store **key-value pairs** similar to dictionaries.

```
mapping(address => uint) balances;
```

Solidity -Control Structures

Solidity supports standard programming control statements.

If-Else

```
if(x == 0)
y = 0;
else
z = 1;
```

While Loop

```
while(x > 0)
{
z++;
}
```

Do While

```
do{
x++;
}while(z > 1);
```

For Loop

```
for(uint x=0; x<=10; x++)
{
z++;
}
```

Solidity

Events

- Events allow smart contracts to **log information to the blockchain**.
- They help external applications detect contract activity.

```
event ValueEvent(bool result);

emit ValueEvent(true);
```

Inheritance

- Solidity supports **object-oriented inheritance**.
- Example

```
contract A
{
uint price = 20;
}
contract B is A
{
}
```

Functions in Solidity

- Functions define **contract behavior**.
- General syntax:

```
function functionName(parameters)
visibility returns(type)
{
}
```

- Example

```
function getValue() public returns(uint)
{
return 10;
}
```

Function Visibility

Functions can have four visibility levels.

Modifier	Description
public	accessible everywhere
external	callable from outside
internal	accessible in derived contracts
private	only within the same contract

Function Modifiers

- Modifiers change function behavior.

Modifier	Purpose
view	cannot modify state
pure	cannot read or modify state
payable	allows receiving Ether
constant	old keyword similar to view

Fallback Function

- Fallback function executes when:
- Ether is sent to the contract
- No matching function exists
- If not defined, Ether transfer will **fail**.
- Example

```
function() payable {
}
```

Constructor Function

- Constructor runs **only once when the contract is deployed**.
- Used to initialize contract variables.
- Example

```
constructor() public {
}
```

Function Modifiers

- Modifiers act like **guards for functions**.
- Used for **security checks**.
- Example

```
modifier onlyOwner {
require(msg.sender == owner);
_;
}
```

References:

1. Imran Bashin, Mastering Block chain, 2nd edition, 2018 packt publishing

2. Aravind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller and steven Goldfeder, Bitcoin and crypto currency Technologies: A comprehensive Introduction, Princeton University Press Metamask - S8class26

Assignment-ll

- Alternative Coins (Altcoins)
- Wallets - Cold wallets - Hardware, Hot wallets -Online, Mobile, Software
- Evolution of Ethereum versions and major protocol upgrades

- Using Solidity language, create a simple bank contract that allows a user to deposit, withdraw and view balance.